



Université De La Manouba
École Nationale Des Sciences De l'informatique



RAPPORT DU PROJET DE CONCEPTION ET DÉVELOPPEMENT

ID : 09 Version : 01

**Sujet : Apprentissage automatique pour la détection et la
prédiction des feux de végétation**

Elaboré par :

MR. WASSIM KHARRAT

MR. RAED FEFEL

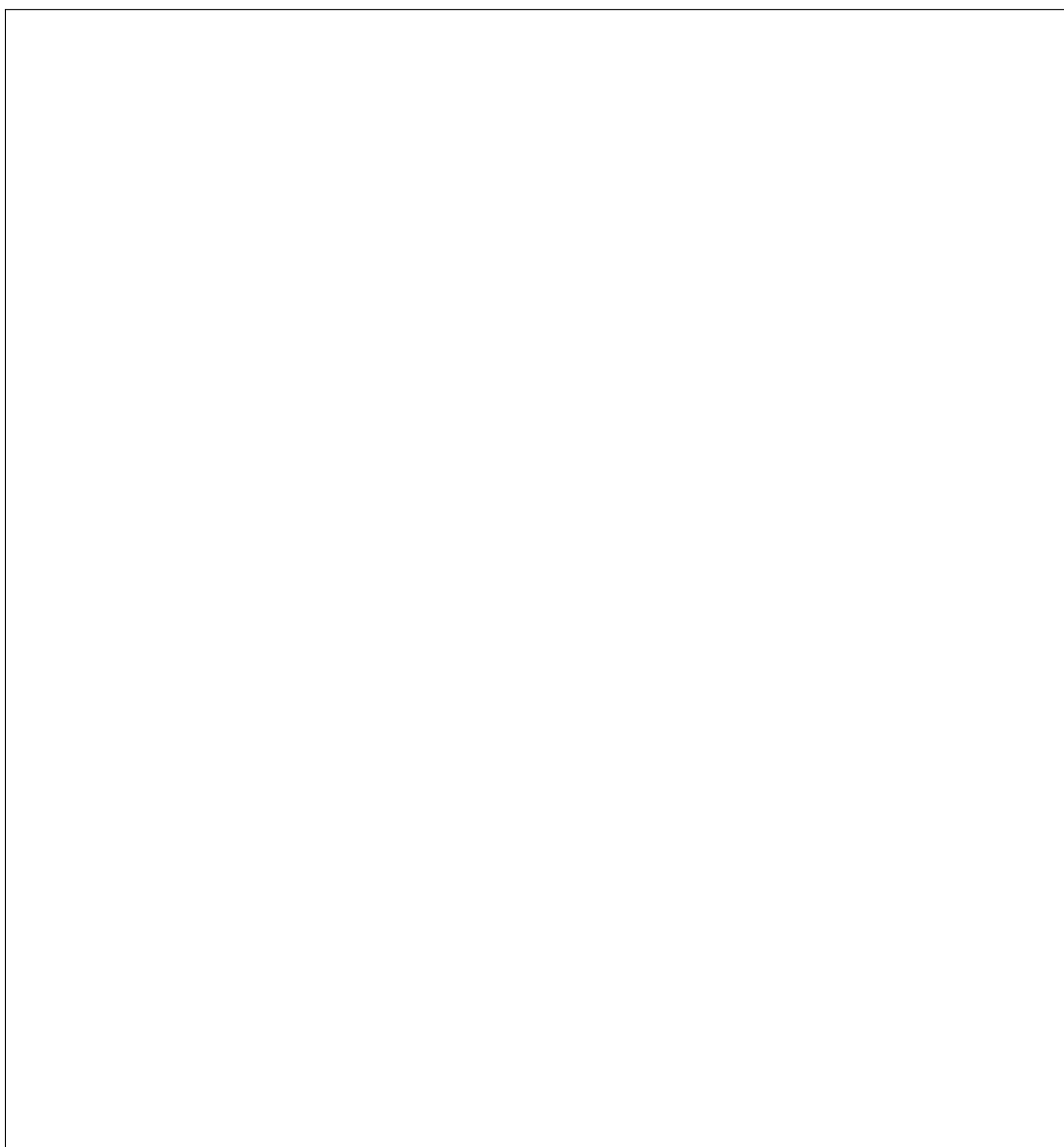
MR. MOUROU MOHAMED AMINE

Encadré par :

DR. CHIRAZ HOUAIDIA

Année Universitaire : 2023 / 2024

Appréciations et signature de l'encadrant

A large, empty rectangular box with a thin black border, intended for the supervisor's appreciations and signature.

Remerciements

Nous souhaitons exprimer notre sincère gratitude à tous les membres de l'équipe pour leur dévouement et leur contribution exceptionnels tout au long du projet de Conception et de Développement. Leur engagement et leur expertise ont été des éléments essentiels de notre succès.

Nous tenons aussi à remercier notre chère encadrante, Dr. Chiraz HOUAIDIA pour sa présence continue, son support et encouragement. Son dévouement et sa collaboration efficace et rapide sont primordiales à la réalisation de ce projet et de ce rapport.

Nous tenons également à remercier nos partenaires et collaborateurs externes pour leur soutien précieux, qui a grandement contribué à l'accomplissement de nos objectifs.

Ce projet a été une véritable démonstration de ce que nous pouvons accomplir lorsque nous travaillons ensemble avec passion, détermination et un esprit d'équipe inégalé.

Nous sommes reconnaissants d'avoir eu l'opportunité de collaborer avec une équipe aussi talentueuse et dévouée. Leurs efforts remarquables ont été essentiels pour atteindre nos objectifs et réaliser quelque chose d'extraordinaire.

Nous anticipons avec enthousiasme la continuation de cette collaboration fructueuse dans de futurs projets.

Table des matières

Introduction Générale	1
1 Contexte et Étude Préliminaire	3
1.1 Contexte et Problématique	3
1.2 Solution proposée	4
1.3 Méthodologie de travail	4
1.4 Concepts de base	5
1.4.1 Apprentissage supervisé VS. Apprentissage non-supervisé	5
1.4.2 Régression VS. Classification	6
1.5 Les algorithmes de classification	8
1.5.1 Soft Margin Classifier :	8
1.5.2 K-Nearest Neighbors	9
1.5.3 Régression Logistique	10
1.5.4 Arbre de Decision	11
1.5.5 Random Forest	12
1.5.6 AdaBoost Classifier	13
2 Analyse et Spécification des Besoins	16
2.1 Spécification des besoins	16
2.1.1 Identification des acteurs	16
2.1.2 Besoins Fonctionnels	17
2.1.3 Besoins non fonctionnels	17
2.1.4 Besoins du domaine	18
2.2 Modélisation des besoins	18
2.2.1 Diagramme de cas d'utilisation général	18
2.2.2 Modélisation détaillée	19
3 Conception	27
3.1 Description et Conception des données	27
3.1.1 Description du jeu de données	27
3.1.2 Pré-traitement et préparation des données :	29
3.1.3 Entraînement et Évaluation des modèles	34
3.2 Conception logicielle	44
3.2.1 Choix de l'architecture logique	44
3.2.2 Choix de l'architecture physique	47

3.2.3	Conception détaillée	50
3.2.4	Choix technologiques	51
4	Réalisation	53
4.1	Environnement de travail et technologies utilisées	53
4.1.1	Environnement matériel	53
4.1.2	Environnement logiciel	53
4.2	Implémentation et Étude des Performances des Modèles de Prédiction	54
4.2.1	Importation des librairies :	54
4.2.2	Etude du modèle de Régression logistique	55
4.2.3	Étude du Modèle de Classification SVM	57
4.2.4	Etude du modèle KNN	59
4.2.5	Synthèse et choix :	61
4.3	Implémentation du tableau de bord	64
4.3.1	Interfaces communes à tous les utilisateurs	64
4.3.2	Interfaces spécifiques à l’agriculteur	65
4.3.3	Interfaces spécifiques à l’agent de l’ONPC	68
4.4	Chronogramme	70
	Conclusion Générale et Perspectives	72
	Bibliographie	74
	Netographie	75

Table des figures

1	Les facteurs qui contrôlent les feux de végétation [4].	1
1.1	Méthodologie incrémentale de développement	5
1.2	Exemple d'illustration avec $K=3$	9
2.1	Diagramme des cas d'utilisation général	19
2.2	Diagramme des cas d'utilisation associé à l'agriculteur	20
2.3	Diagramme de séquences relatif au cas d'utilisation "Afficher les graphes de prédiction"	21
2.4	Diagramme des cas d'utilisation spécifique au cas d'utilisation "gérer le compte"	21
2.5	Diagramme de séquences relatif au cas d'utilisation "gérer le compte"	22
2.6	Diagramme de séquences relatif au cas d'utilisation "Visualiser les incendies"	23
2.7	Diagramme des cas d'utilisation relatif à l'agent de l'ONPC	24
2.8	Diagramme de séquences relatif au cas d'utilisation "Afficher les données des agriculteurs"	25
2.9	Diagramme de séquences relatif au cas d'utilisation "Localiser les agriculteurs et leurs champs"	26
3.1	Extrait du jeu de données	28
3.2	Extrait du jeu de données après l'encodage	31
3.3	Résultat de mise en échelle sur les 3 attributs du jeu de données	32
3.4	Détection des valeurs aberrantes à travers le tracé de boîte.	33
3.5	Résultat du nettoyage des valeurs aberrantes	34
3.6	Matrice de Confusion	37
3.7	Création d'un jeu de données <i>Bootstrap</i>	43
3.8	<i>Stacking Model</i>	44
3.9	Architecture en Flots des données [URL10]	45
3.10	Architecture centrée sur les données [URL11]	45
3.11	Architecture 2-tiers [URL12]	47
3.12	Architecture 3-tiers [URL12]	49
3.13	Diagramme de classe	51
4.1	Courbe d'étalonnage du 1er modèle	57
4.2	Courbe d'étalonnage du 2ème modèle	59
4.3	Courbe d'étalonnage du 3ème modèle	61
4.4	Structure du modèle final adopté	62
4.5	Courbe d'étalonnage	63

4.6	Matrice de confusion	63
4.7	Interprétation graphique des résultats de performances	64
4.8	Interface d'authentification	65
4.9	Erreur d'authentification	65
4.10	Interface d'accueil de l'agriculteur	66
4.11	Interface d'accueil de l'agriculteur dans le cas détection d'un incendie	67
4.12	Interface de gestion de compte	68
4.13	Interface d'accueil de l'agent de l'ONPC	69
4.14	Interface d'investigation et d'historique	70
4.15	Chronogramme de travail	70

Table des *listing*

1	Extraction de la cible et des attributs	29
2	Distribution des données selon leurs classes	29
3	Encodage de la variable cible	30
4	Normalisation	32
5	Détection et suppression des duplicatas et des valeurs manquantes	33
6	Détection et suppression des valeurs aberrantes	34
7	Validation croisée d'un modèle donné	35
8	Matrice de confusion	37
9	Courbe Precision-Recall et courbe ROC	39
10	Implémentation de l'Optuna	40
11	Courbe d'étalonnage	42
12	Importation des librairies	54
13	Ajustement des hyperparamètres du modèle de régression logistique	55
14	Étalonnage du modèle 1	56
15	Évaluation du modèle SVC	57
16	Ajustement des hyperparamètres du modèle SVC	58
17	Étalonnage du modèle2	58
18	Ajustement des hyperparamètres du modèle KNN	60
19	Étalonnage du modèle KNN	60

Introduction Générale

Selon un récent rapport de Thomson Reuters [URL1], une vague de chaleur et des incendies endommagent gravement la récolte céréalière tunisienne. En effet, les cultures subissent des dommages directs à cause des températures élevées, qui ont déjà atteint 49 degrés Celsius (117 degrés Fahrenheit) cet été. De plus, une grande partie du reste de cette récolte est perdue à cause des incendies qui font rage au pays dès les premiers mois de l'été. D'autre part, des travaux d'investigation [1][4] sur cette problématique ont rapporté que les feux de végétation sont contrôlés par les interactions entre trois facteurs : le climat, la végétation et les actions humaines 1. La plupart des régions du globe sont affectées par des feux, mais les caractéristiques de ces derniers varient. C'est pourquoi on s'intéresse à leurs régimes, un terme qui recouvre à la fois le comportement des feux, leur nombre par an, leur taille moyenne, leur continuité spatiale, leur type dominant (feu de surface ou feu de cimes) et leurs impacts sur une région ou sur un écosystème donné.

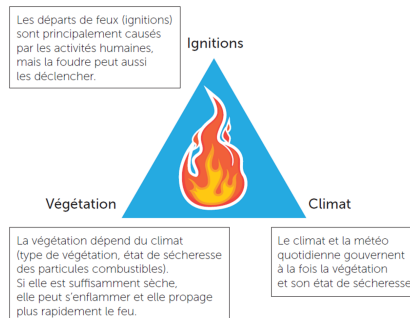


FIGURE 1 – Les facteurs qui contrôlent les feux de végétation [4].

Pour ces raisons, la prévision et la cartographie du risque d'incendies constituent une priorité absolue en matière de recherche dans les champs sujettes aux incendies à travers le pays afin de préserver les récoltes et assurer la souveraineté alimentaire.

Pour anticiper et découvrir les incendies, plusieurs technologies et techniques ont été mises en avant. Pour prévoir la probabilité d'incendies de forêt et évaluer le risque de dommages induits par ces feux, les techniques d'intelligence artificielle et d'apprentissage automatique suscitent beaucoup d'intérêt et s'imposent comme une solution cruciale du fait de leurs résultats prometteurs dans ce domaine.

Cependant, malgré un état de l'art qui foisonne de travaux sur la détection et prédiction des feux [1][2][3], la plupart de ces travaux sont ciblés sur les feux de forêts et ne sont pas aussi efficaces pour les champs de blé en raison de l'absence d'images relatives aux récoltes céréalières, des caractéristiques hautement inflammables du terrain et de la vitesse

de propagation du feu [2].

Dans ce travail, l'objectif principal consiste à créer un système de prédiction des feux dans la récolte céréalière tunisienne qui va permettre d'anticiper les incendies dans les champs de végétation en général et les champs de blé en particulier, et ce, en mettant en place un système fiable et efficace de suivi, d'investigation et d'alerte. En effet, ce système permettra en premier lieu, le suivi en temps réel de données météorologiques et d'informations pertinentes liées à la végétation telles que l'indice NDVI ou Normalized Difference Vegetative Index et les anomalies thermiques. L'analyse de ces données, devrait permettre au système de déduire l'état de la récolte et de dresser une cartographie de risque et ainsi identifier les positions éventuelles des feux déclenchés ainsi qu'une estimation des dommages engendrés. Il fournit également des alertes en cas de risque confirmé de feu ce qui garantit une intervention rapide et efficace des parties concernées. Nous proposons, aussi, de développer un tableau de bord permettant à l'utilisateur d'interagir avec le système, se renseigner et suivre en temps réel l'état de ses champs.

Le présent rapport détaille les différentes étapes qui ont permis l'aboutissement de ce travail. Dans un premier chapitre *Etude préliminaire*, nous présentons le contexte du travail et nous passons en revue les différents concepts et techniques liés à notre travail particulièrement les aspects de Machine Learning.

Ensuite, nous consacrons le deuxième chapitre à l'analyse et la spécification des besoins fonctionnels et non fonctionnels de notre système. Cela servira comme point de référence lors de la réalisation du système. Cette partie contient les définitions des acteurs, leurs besoins et quelques diagrammes décrivant les différents cas d'utilisation pour les différents acteurs et les différents scénarios d'utilisation.

Le troisième chapitre *Conception* est dédié à la conception des données sur lesquelles nous allons travailler en détaillant les aspects de pré-traitement et de nettoyage du jeu de données. De plus, ce chapitre contient également une conception logicielle mettant en avant l'architecture logique et physique adoptées ainsi qu'un diagramme de classe.

Les étapes de réalisation de notre système ainsi qu'une évaluation succincte de ses performances feront l'objet du cinquième chapitre *Réalisation*.

Nous clôturons ce présent rapport par une conclusion et quelques perspectives.

Chapitre 1

Contexte et Étude Préliminaire

Introduction

Dans ce premier chapitre, nous présentons le contexte du travail et nous réalisons une étude préliminaire des concepts clés qui serviront à l'élaboration de ce travail.

1.1 Contexte et Problématique

La problématique des feux dans les champs de végétation est un enjeu important dans de nombreuses régions du monde, notamment dans les zones agricoles sujettes aux incendies de cultures. Ces feux peuvent causer des dommages considérables aux cultures, à l'environnement et aux infrastructures, ainsi que des pertes économiques pour les agriculteurs et les communautés locales. Les impacts de cette problématique peuvent être répartis comme suit :

- Impacts environnementaux :
 - Destruction de la flore et de la faune
 - Perte de la biodiversité
 - Érosion des sols
 - Pollution de l'air et émission de gaz à effet de serre
- Impacts économiques :
 - Pertes de récoltes
 - Dégâts matériels
 - Coûts élevés de lutte contre les incendies et de reforestation
- Impacts sur la santé humaine :
 - Risques d'intoxication par les fumées
 - Blessures et décès
 - Augmentation des problèmes respiratoires

Ces feux dans les champs de végétation sont souvent causés par des facteurs naturels liés généralement aux caractéristiques de la végétation en question et aux données météorologiques (sécheresse, foudre) ou humains liés aux pratiques agricoles (négligence, criminalité).

Face à l'ampleur de cette problématique, le développement de systèmes de prédiction des feux de champs de végétation s'avère crucial. Ces systèmes permettraient d'identifier les zones à risque et d'anticiper leurs déclenchements.

Plusieurs systèmes de prédiction des feux de champs de végétation existent déjà, tels que :

- **MODIS** : un système satellitaire de la NASA qui permet de détecter les départs de feu et de suivre leur évolution. Cependant, ce système peut être saturé par la présence de nuages ou de fumées, ce qui peut limiter sa capacité à détecter les feux [URL6].
- **FIRMS** : un système développé par l'Université du Maryland qui utilise des données satellitaires et des algorithmes de détection d'incendie pour produire des cartes en temps réel des feux actifs mais il est principalement axé sur la détection des feux actifs et offre des capacités limitées en matière de prédiction [URL5].
- **PROMET** : un modèle français de prévision du risque d'incendie de forêt qui prend en compte des facteurs météorologiques, topographiques et liés à la végétation. Cependant, ce modèle français est principalement adapté aux forêts méditerranéennes et peut ne pas être performant dans d'autres types d'écosystèmes [5][6].

1.2 Solution proposée

En guise de solution à la problématique identifiée, et dans le but de développer un système spécifique aux champs de végétation en général et de blé en particulier, nous proposons de créer un système piloté par des techniques d'intelligence artificielle, pour la prédiction des feux dans les champs de végétation, et ce, en mettant en place un moyen fiable et efficace de suivi, d'investigation et d'alerte. En effet, ce système permettra en premier lieu, le suivi en temps réel de données météorologiques et d'informations pertinentes liées à la végétation telles que l'indice NDVI ou Normalized Difference Vegetative Index et les anomalies thermiques. Ces données devraient alimenter un modèle prédictif pour donner lieu à un état précis de la récolte et de dresser une cartographie de risque et ainsi identifier les positions éventuelles des feux déclenchés ainsi qu'une estimation des dommages engendrés. Il fournit également des alertes en cas de risque confirmé de feu ce qui garantit une intervention rapide et efficace des parties concernées. Nous proposons, aussi, de développer un tableau de bord permettant à l'utilisateur d'interagir avec le système, se renseigner et suivre en temps réel l'état de ses champs.

1.3 Méthodologie de travail

Dans le cadre de notre projet, nous avons adopté une méthodologie de prototypage incrémental pour guider notre processus de développement. Cette méthodologie a été choisie en raison de sa flexibilité, de son adaptabilité et de sa capacité à favoriser la collaboration avec les utilisateurs finaux. La méthodologie de prototypage incrémental, tel qu'illustré sur la figure 1.1 repose sur un cycle itératif de développement et d'amélioration du système. Voici comment nous avons appliqué cette méthodologie dans notre projet :

1. Analyse des besoins : Nous avons commencé par identifier et définir les besoins du projet en étroite collaboration avec les utilisateurs finaux et les parties prenantes. Cette

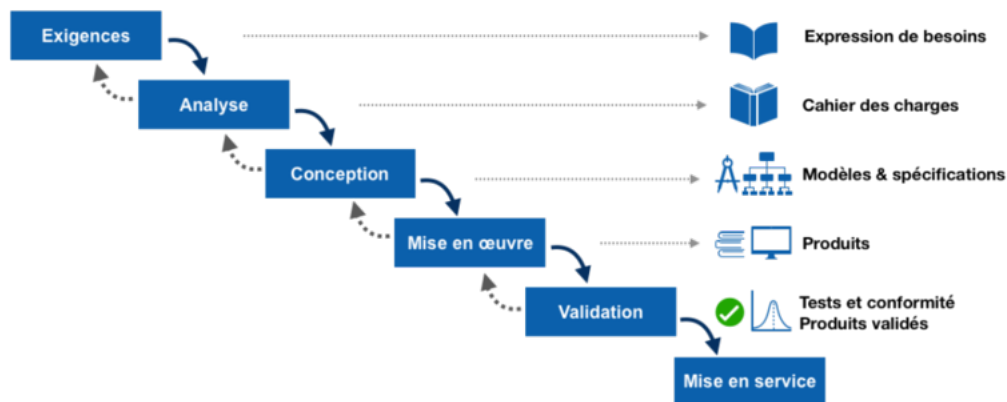


FIGURE 1.1 – Méthodologie incrémentale de développement

analyse approfondie nous a permis de déterminer les fonctionnalités clés qui devaient être incluses dans le premier prototype.

2. Conception du prototype initial : À partir des exigences identifiées, nous avons développé un prototype initial mettant l'accent sur les fonctionnalités principales du système. Ce prototype était conçu pour être fonctionnel plutôt que d'être axé sur l'aspect visuel, en utilisant les outils et les technologies appropriés.
3. Évaluation du prototype : Nous avons réalisé une première évaluation du prototype initial afin d'identifier les points faibles et les améliorations nécessaires pour les itérations ultérieures.
4. Validation et intégration : À chaque nouvelle itération, le prototype a été amélioré avec de nouvelles fonctionnalités ou un nouveau design puis intégrées au système global, en veillant à maintenir la cohérence et la compatibilité avec les itérations précédentes.

1.4 Concepts de base

Nous commençons par expliquer quelques concepts clés liés aux techniques de Machine Learning et qui nous guideront dans nos choix technologiques par la suite.

Lors de la réalisation de ce projet, l'utilisation des modèles d'apprentissage automatique nous sera non seulement utile mais plutôt primordiale afin d'assurer les fonctions de notre système. Nous dressons, donc, une analyse comparative des différents types de modèles ML tout en soulignant leurs avantages, inconvénients et leurs cas d'application appropriés.

1.4.1 Apprentissage supervisé VS. Apprentissage non-supervisé

Apprentissage Supervisé : L'apprentissage supervisé est une approche qui utilise des jeux de données (fréquemment nommés ici "datasets") labélisés. Ces datasets sont créés de ma-

nière à pousser l'algorithme à classer les données et à prédire les classes prédéfinies d'une manière précise. Les problèmes d'apprentissage supervisé peuvent être séparés en deux types de problème : Classification ou Régression [URL4].

Apprentissage Non-Supervisé : L'apprentissage non-supervisé repose sur des jeux de données non labélisés, c-à-d, ces données n'ont pas des catégories préexistantes. Ils découvrent les tendances sans avoir besoin d'une intervention externe et divise les données automatiquement en classes selon des caractéristiques communes. La qualité d'une méthode de classification est d'ailleurs mesurée par sa capacité à découvrir certains ou tous les motifs cachés. Nous distinguons entre 3 types d'apprentissage non supervisé :

- **Clustering** : il s'agit de ressortir des similarités afin de regrouper des données d'une façon autonome.
Nous citons à titre d'exemple les modèles *K-means* et *Clustering Hiérarchique*
- **Association** : Elle utilise plusieurs règles afin de définir les relations entre les variables dans un dataset donné.
Nous citons à titre d'exemple *Algorithme à priori*, *Algorithme Eclat* et *Algorithme FP-Growth*.
- **Réduction de dimension** : Elle est utilisée quand le nombre de caractéristiques ou *Features* dans un dataset est très élevé. Elle réduit le nombre des caractéristiques à une taille utilisable tout en gardant l'intégrité des données.
Afin d'évaluer la performance des modèles d'apprentissage non supervisé, des métriques sont introduites tel que :
 - *silhouette-score* : ce coefficient indique à quel point un objet est agrégé avec son propre cluster et séparé des autres clusters, sa valeur varie entre -1 et 1.
 - *score de Calinski-Harabasz* : mesure la séparation entre les clusters et la compacité à l'intérieur de chaque cluster, sa valeur varie de 0 jusqu'à l'infini.
 - *indice de davies-dublin* : mesure la similitude moyenne de chaque cluster avec le cluster qui lui ressemble le plus, sa valeur varie de 0 jusqu'à l'infini.
 - *indice de rand ajusté* : mesure la similarité entre 2 clusters, elle varie entre -1 et 1.
 - *inertia* : mesure la cohésion entre les points à l'intérieur de chaque cluster.

Dans notre projet, nous utilisons des algorithmes supervisés étant donné que le jeu de données adopté possède des labels et deux classes prédéfinies (fire, no-fire). Pour cela, nous détaillons, dans la section suivante, les algorithmes d'apprentissage supervisé que nous pouvons utiliser afin d'éclairer nos choix.

1.4.2 Régression VS. Classification

Régression : Les algorithmes de régression sont des algorithmes d'apprentissage automatique qui prévoient la valeur d'une variable dépendante pour des nouvelles entrées non-connues au préalable. Ces algorithmes modélisent la relation entre les caractéristiques des entrées et la valeur cherchée en sortie. Ils existent plusieurs types d'algorithmes de régression, à savoir :

- **La régression simple** : utilisée pour prédire une variable continue dépendante à partir d'une seule variable indépendante

- **La régression multiple** : utilisée pour prédire une variable continue dépendante à partir de plusieurs variables indépendantes
- **La régression non-linéaire** : utilisée quand la relation entre les variables indépendantes en entrée et la variable dépendante de sortie est non-linéaire.

L'analyse de régression est utilisée dans divers domaines, notamment l'économie, la finance, le marketing et la psychologie, pour comprendre et prédire les relations entre différentes variables. Elle est utilisée pour faire des prédictions, identifier les tendances et comprendre les mécanismes sous-jacents qui alimentent les données.

Par exemple, dans un modèle de régression linéaire, l'objectif peut être de prédire le prix d'une maison en fonction de sa taille, de son emplacement et d'autres caractéristiques. La taille de la maison et son emplacement seraient les variables indépendantes, et le prix de la maison serait la variable dépendante.

Classification : Les algorithmes de classification sont des algorithmes d'apprentissage automatique dont le but est de prédire une classe pour une donnée en entrée. En classification, le modèle est entièrement entraîné en utilisant des données d'entraînement. Ensuite, l'efficacité de ce modèle sera évalué en utilisant des données de test, sachant que ces données d'entraînement et de test doivent obligatoirement être distinctes afin de pouvoir tester les performances du modèle sur de nouvelles entrées.

Nous identifions deux types d'algorithmes de classification :

- Des algorithmes ayant besoin de préparer le modèle au préalable à partir des données d'entraînement avant de réaliser toute autre prédiction sur des données futures. Ils passent plus de temps pendant la phase d'entraînement ce qui permet une meilleure généralisation des poids et réduit le temps de prédiction. Nous prenons comme exemple le modèle de Régression Linéaire ou le modèle Support Vector Machine (SVM).
- Des algorithmes qui ne créent pas un modèle directement des données d'entraînement mais mémorisent ces données et à chaque fois où ils doivent réaliser une prédiction, ils cherchent le voisin le plus proche comme le "K-nearest Neighbor"

Afin de pouvoir comparer ces deux approches d'apprentissage supervisé et réaliser le bon choix entre elles, nous nous référons au tableau suivant qui met en valeur les principales différences entre les algorithmes de régression et les algorithmes de classification :

<i>Algo. de Classification</i>	<i>Algo. de Régression</i>
La variable de sortie est discrète .	La variable de sortie est continue .
L'objectif est de prédire une catégorie/label d'une classe	L'objectif est de prédire une valeur numérique continue
Dans ces algorithmes, nous essayons de trouver la meilleure frontière de décision qui va réussir à séparer deux classes ou plus avec la plus grande séparation possible.	Dans ces algorithmes, nous essayons de trouver la meilleure droite qui va représenter les tendances des données.
Les métriques d'évaluation qui peuvent être utilisées sont Precision, Recall, accuracy, AUC-ROC, F1-score et la matrice de confusion .	Les métriques d'évaluation peuvent être Square Mean Error, R2-score ou MAPE

Etant donné notre objectif qui consiste à déterminer s'il y a feu dans un champ de végétation donné ou pas et en tenant compte de notre jeu de données, nous optons pour l'approche de classification comme solution d'apprentissage supervisé et nous détaillons par la suite les différents algorithmes existants afin de mieux cerner nos choix.

1.5 Les algorithmes de classification

1.5.1 Soft Margin Classifier :

Cet algorithme est conçu pour effectuer la tâche de classification et consiste à ramener un problème de classification à un hyperplan (feature space) dans lequel les données sont séparées en plusieurs classes dont la frontière est la plus éloignée possible des points de données.

Terminologie :

- *Hyperplan* : Un classificateur linéaire qui sépare les données et maximise la distance entre les classes.
- *Vecteurs de soutien* : Les points les plus proches qui seuls sont utilisés pour la détermination de l'hyperplan.
- *Marge* : Le maximum de la distance minimale entre un hyperplan et les exemples d'apprentissage
- *Noyau (kernel)* : Une méthode qui consiste à utiliser un classificateur linéaire pour résoudre un problème non-linéaire
- *Hinge loss* : c'est une fonction de perte qui maximise l'intervalle entre les classes, la rendant très efficace pour les SVM.

Avantages :

- + Efficacité dans des espaces à grande dimension
- + Capacité à travailler dans un espace de données non linéaire
- + Utilisation d'un sous-ensemble de données dans la fonction de décision (*vecteurs de supports*) ce qui le rend efficace en termes de mémoire
- + Bonne généralisation

Inconvénients :

- Sensibilité à l'échelle : les caractéristiques doivent être mises à l'échelle sinon pour des caractéristiques à valeurs très élevées ou très basses peuvent avoir un effet néfaste en terme de précision.
- Sensibilité aux hyperparamètres
- Résultats difficiles à interpréter
- Complexité de calcul : Ce modèle est coûteux en termes de temps.

Applications :

- ◇ Détection de visage
- ◇ Catégorisation des textes

1.5.2 K-Nearest Neighbors

Il s'agit de l'un des algorithmes de classification les plus basiques et connus, le principe de cet algorithme est d'affecter un point non classé appartenant à un jeu de données au groupe renfermant la majorité de ses plus proches voisins. L'intuition derrière cet algorithme est plus simple que celles des autres algorithmes :

1. Choisir un nombre **K**
2. Calculer la distance entre le point à classer et les autres points du nuage en utilisant soit la distance euclidienne (equation 1.1) soit la distance de Manhattan (equation 1.2).

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad (1.1)$$

Formule de la distance euclidienne

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n |q_i - p_i| \quad (1.2)$$

Formule de la distance de Manhattan

3. Parmi les points et selon la distance choisie, choisir les **K** points plus proches.
4. Parmi ces **K** points, compter les points appartenant à chaque catégorie.
5. Affecter le point à la catégorie la plus représentée dans l'ensemble de ces **K** points.

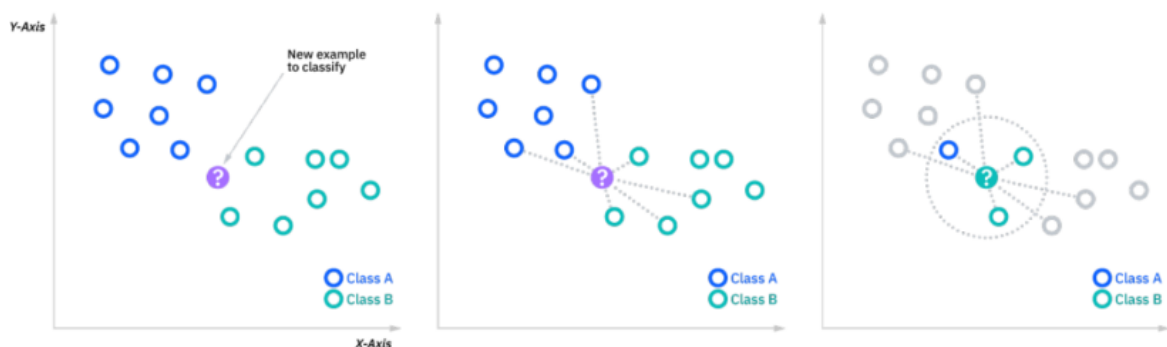


FIGURE 1.2 – Exemple d'illustration avec K=3

Avantages :

- + Facile à implémenter

- ✦ Adaptation : Ce modèle stocke les données en mémoire et s'ajuste à chaque fois qu'une nouvelle donnée s'ajoute sans la nécessité d'une phase d'apprentissage.
- ✦ Non paramétrique : c-à-d qu'il ne nécessite pas une distribution sous-jacente des données.

Inconvénients :

- Mise à l'échelle difficile : KNN est un modèle "paresseux" puisqu'il stocke les données et utilise la mémoire plus que les autres modèles, ceci est coûteux en terme de temps et nécessite beaucoup plus de puissance de calcul, d'ailleurs, certaines structures de données tel que **ball-tree** ont été instaurées pour remédier à ce problème.
- Non flexible avec la dimension : Ce qui signifie qu'il peut fonctionner mal avec des espaces à grandes dimensions et ce phénomène est connu par *le phenomne de peak* et il peut commettre plus d'erreurs lorsque la taille de l'échantillon est petite.
- Enclin au sur-ajustement(Overfitting)

Applications :

- ◇ Moteurs de recommandation : en utilisant les données de parcours de navigation des sites Web, l'algorithme KNN a été utilisé pour fournir des recommandations automatiques aux utilisateurs sur du contenu supplémentaire.
- ◇ Prétraitement des données : Cet algorithme peut être utilisé comme méthode d'imputation pour traiter le problème des données manquantes dans un jeu de données.
- ◇ Reconnaissance de formes.

1.5.3 Régression Logistique

Notons d'abord que pour la régression logistique, on dispose de deux types de variables : les variables dépendantes qui sont les variables cibles et les variables indépendantes qui sont les caractéristiques. La régression logistique est un modèle statistique utilisé pour effectuer la classification binaire des données en utilisant la fonction logistique **sigmoïde**. Cette dernière prend les entrées comme variables indépendantes et produit une probabilité qui prédit la classe ou bien la valeur des variables dépendantes selon la position de cette valeur par rapport à la valeur du seuil. En général, on distingue trois types de régression logistique :

- **binaire** : où les variables dépendantes du modèle ne peuvent avoir que 2 valeurs possibles **1** ou **0** modélisant par exemple «succès» et «échec» ou «oui» et «non».
- **multinomial** : où les variables dépendantes peuvent avoir plus que 2 types non ordonnées, c-à-d, n'ayant aucune signification en termes d'ordre.
- **ordinal** : où les variables dépendantes du modèle peuvent avoir plus que 2 types ordonnées par exemple «faible», «moyen» et «fort».

Hypothèses :

- Les variables caractéristiques doivent être indépendantes et il n'y a aucune corrélation entre elles.
- En cas de régression logistique, les variables cibles doivent être binaires.
- La relation entre les variables dépendantes et les variables indépendantes doit être linéaire.
- Aucune valeur aberrante.
- Échantillon de grande taille.

Avantages :

- + Facilité d'implémentation
- + Adaptabilité aux classes discrètes : il n'exige aucune hypothèse sur la distribution des classes.
- + il garantit une bonne précision pour de nombreux ensembles de données simples et fonctionne bien lorsque l'ensemble de données est linéairement séparable.
- + La régression logistique est moins encliné au sur-ajustement, mais elle peut sur-ajuster dans des ensembles de données de grande dimension. On peut envisager des techniques de régularisation (L1 et L2) pour éviter le sur-ajustement dans ces scénarios.

Inconvénients :

- La principale limitation de la régression logistique est l'hypothèse de linéarité entre la variable dépendante et les variables indépendantes.
- Les problèmes non linéaires ne peuvent pas être résolus avec la régression logistique car elle a une surface de décision linéaire.
- Il est difficile d'obtenir des relations complexes en utilisant la régression logistique. Des algorithmes plus puissants et compacts tels que les réseaux de neurones peuvent facilement surpasser cet algorithme

Applications :

- ◇ Détection des maladies
- ◇ Maintenance prédictive, c-à-d, prédire l'état du fonctionnement des machines en fonction des caractéristiques.

1.5.4 Arbre de Decision

Un arbre de décision est un outil visuel qui permet de représenter un processus décisionnel et ses différentes options. Il se compose de nœuds et de branches, où chaque nœud représente une décision et chaque branche représente un choix possible. Les branches se terminent par des feuilles, qui représentent les résultats possibles des différentes décisions.

Avantages :

- + Simplicité et clarté : Les arbres de décision sont faciles à comprendre et à interpréter.
- + Visualisation efficace : La représentation graphique permet de visualiser clairement les différentes options et leurs conséquences.
- + Aide à la prise de décision : Les arbres de décision permettent de structurer le processus décisionnel et d'identifier les choix les plus pertinents en fonction des objectifs et des contraintes.
- + Flexibilité : Les arbres de décision peuvent être adaptés à des situations complexes et à de nombreux domaines d'application.

Inconvénients :

- Simplicité excessive : Pour des situations complexes, les arbres de décision peuvent ne pas capturer toutes les nuances et les interactions entre les facteurs.
- Données quantitatives : La construction d'un arbre de décision nécessite souvent des données quantitatives pour évaluer les probabilités et les coûts associés à chaque choix.
- Difficulté d'interprétation : Pour des arbres de décision complexes, il peut être difficile de comprendre le cheminement qui a conduit à une décision particulière.

Applications :

- ◇ Aide à la décision stratégique : Les arbres de décision peuvent être utilisés pour évaluer différentes options stratégiques, telles que le lancement d'un nouveau produit, l'entrée sur un nouveau marché ou l'investissement dans une nouvelle technologie.
- ◇ Diagnostic médical : Les arbres de décision peuvent être utilisés pour aider les médecins à diagnostiquer des maladies en fonction des symptômes et des facteurs de risque des patients.
- ◇ Gestion des risques : Les arbres de décision peuvent être utilisés pour identifier et évaluer les risques potentiels dans un projet ou une entreprise.

1.5.5 Random Forest

Les forêts aléatoires (Random Forest) sont un ensemble d'arbres de décision agrégés. En d'autres termes, il s'agit d'une méthode d'apprentissage automatique qui combine plusieurs arbres de décision pour obtenir un modèle de prédiction plus robuste et performant.

Pour construire une forêt aléatoire, on suit les étapes suivantes :

1. Générer plusieurs arbres de décision : On crée un grand nombre d'arbres de décision en utilisant un sous-ensemble aléatoire des données d'apprentissage et en choisissant aléatoirement un sous-ensemble de caractéristiques à chaque nœud de l'arbre.
2. Agrégation des prédictions : Pour une nouvelle instance à prédire, on fait passer l'instance dans chacun des arbres de décision de la forêt et on obtient une prédiction de chaque arbre. La prédiction finale de la forêt aléatoire est obtenue en agrégeant les prédictions individuelles des arbres, par exemple en prenant la moyenne pour la régression ou le vote majoritaire pour la classification.

Avantages :

- + Précision accrue : Les forêts aléatoires sont généralement plus précises que les arbres de décision individuels car elles réduisent l'impact du sur-ajustement et de la sensibilité aux valeurs aberrantes.
- + Robustesse : Les forêts aléatoires sont moins sensibles au bruit et aux données manquantes que les arbres de décision individuels.
- + Interprétabilité : Les forêts aléatoires permettent d'identifier les caractéristiques les plus importantes pour la prédiction en analysant l'importance relative de chaque caractéristique dans l'ensemble des arbres.
- + Flexibilité : Les forêts aléatoires peuvent être utilisées pour des tâches de classification et de régression.

Inconvénients :

- Coût de calcul : La construction d'une forêt aléatoire peut être plus longue que la construction d'un seul arbre de décision en raison du grand nombre d'arbres à générer.
- Boîte noire : L'interprétation des prédictions individuelles d'une forêt aléatoire peut être plus difficile que pour un seul arbre de décision.

Applications :

- ◇ Classification : Les forêts aléatoires sont couramment utilisées pour des tâches de classification, telles que la reconnaissance d'images, la détection de fraude et le diagnostic médical.
- ◇ Régression : Les forêts aléatoires peuvent également être utilisées pour des tâches de régression, telles que la prédiction du prix d'une maison, la prédiction de la demande pour un produit ou la modélisation du risque de crédit.
- ◇ Sélection de caractéristiques : Les forêts aléatoires peuvent être utilisées pour identifier les caractéristiques les plus importantes dans un ensemble de données, ce qui peut être utile pour la réduction de dimension et la sélection de variables.

1.5.6 AdaBoost Classifier

Le classifieur AdaBoost est un algorithme d'apprentissage automatique de type "ensemble learning" utilisé pour les tâches de classification. Il combine plusieurs modèles dits "faibles apprenants" (souvent des arbres de décision simples) en les entraînant séquentiellement pour obtenir un modèle de prédiction final plus performant.

Son fonctionnement inclut les étapes suivantes :

1. Initialisation : Tous les points de données dans l'ensemble d'apprentissage reçoivent un poids égal.
2. Entraîner un faible apprenant : Un premier faible apprenant est entraîné sur l'ensemble des données.
3. Calculer l'erreur : On calcule l'erreur de classification du faible apprenant.

4. Ajuster les poids : Les poids des points de données mal classés par le faible apprenant sont augmentés, forçant le prochain apprenant à se concentrer davantage sur ces points difficiles.
5. Répéter : Les étapes 2 à 4 sont répétées pour un nombre prédéfini d'itérations, chaque nouvel apprenant étant entraîné sur l'ensemble de données pondéré en tenant compte des erreurs des apprenants précédents.
6. Classification finale : Pour une nouvelle instance à prédire, on la présente à tous les faibles apprenants entraînés. La prédiction finale est obtenue en prenant un vote pondéré des prédictions individuelles de chaque apprenant, les poids étant basés sur la performance de chaque apprenant.

Avantages :

- ✦ Amélioration de la précision : En combinant plusieurs faibles apprenants, AdaBoost peut souvent surpasser la performance d'un seul apprenant faible.
- ✦ Robustesse au sur-ajustement : AdaBoost est moins susceptible au sur-ajustement par rapport à des algorithmes d'apprentissage unique car il se concentre sur les points de données mal classés lors de l'entraînement des apprenants suivants.
- ✦ Flexibilité : AdaBoost peut être utilisé avec différents types de faibles apprenants, offrant une certaine adaptabilité aux problèmes de classification variés.

Inconvénients :

- Boîte noire : L'interprétation du fonctionnement interne d'AdaBoost et la compréhension de la contribution de chaque faible apprenant peut être difficile.
- Sensibilité aux valeurs aberrantes : Les points de données aberrants peuvent avoir un impact important sur le processus d'apprentissage d'AdaBoost.
- Coût de calcul : L'entraînement d'AdaBoost peut être plus lent que celui d'un seul apprenant faible en raison des multiples tours d'entraînement nécessaires.

Applications :

- ◇ Classification d'images : AdaBoost est utilisé pour des tâches de classification d'images, comme la reconnaissance d'objets ou la détection de visages.
- ◇ Détection de fraude : AdaBoost peut être utilisé pour identifier les transactions frauduleuses en analysant les caractéristiques des transactions passées.
- ◇ Bioinformatique : AdaBoost est utilisé dans des applications de bioinformatique, comme la classification des protéines ou la prédiction de la structure des protéines.

Pour notre solution proposée, l'objectif serait d'implémenter et de tester quelques algorithmes que nous venons de présenter afin de décider quel algorithme serait le mieux adapté pour notre système de prédiction des feux dans les champs de végétation.

Conclusion

Dans ce chapitre, nous avons commencé par introduire le contexte général de notre travail en identifiant la problématique et la solution proposée. Ensuite, nous avons bien exposé tous les concepts théoriques et techniques nécessaires dans notre projet afin de justifier nos choix futurs, ainsi que notre méthode de travail. Le prochain chapitre, nous entamerons l'étape de spécification des besoins fonctionnels et non fonctionnels.

Chapitre 2

Analyse et Spécification des Besoins

Après avoir introduit notre projet et les différents objectifs que nous traitons, nous devons maintenant identifier et présenter les différents acteurs qui vont participer au fonctionnement de notre système, ainsi que les besoins que nous devons prendre en considération lors du développement du système. Cela va permettre de s'aligner avec les critères déjà fixés dans le cahier des charges fonctionnelles.

Cette partie inclut aussi quelques diagrammes des cas d'utilisation relatifs aux différents acteurs ainsi que des diagrammes de séquence représentant les différents scénarios d'utilisation.

2.1 Spécification des besoins

Les besoins fonctionnels et non fonctionnels définissent les fonctions d'un système logiciel et la façon dont le système doit se comporter lorsqu'il est présenté avec des entrées ou des conditions spécifiques. Ceux-ci peuvent inclure des calculs, la manipulation et le traitement de données ainsi que d'autres fonctionnalités spécifiques.

2.1.1 Identification des acteurs

En terme d'acteurs, nous proposons, principalement, deux types d'utilisateurs de notre système à savoir :

L'agriculteur : Il sera l'utilisateur principal de notre application qui va lui permettre de gérer l'état de ces récoltes. Nous désignons ici l'agriculteur comme étant toute personne ayant des champs de végétation de grande taille (de l'ordre de quelques hectares) et concernée par la surveillance en temps réel de ses récoltes.

Agent de L'ONPC : En sa qualité de représentant d'organisme responsable des interventions qui doivent être réalisées afin de pouvoir contrôler les incendies et les anticiper, l'agent de l'Office National de la Protection Civile est un utilisateur principal de notre système ayant accès aux fonctionnalités de suivi, d'alerte et d'investigation comme annoncé dans nos objectifs.

Nous soulignons aussi l'existence d'un acteur secondaire représenté par **un système de détection** et composé, éventuellement, d'un ensemble de capteurs qui va permettre de collecter les données nécessaires à la détection des feux dans une surface donnée.

2.1.2 Besoins Fonctionnels

Notre système doit satisfaire l'ensemble des besoins fonctionnels suivants, à savoir :

Pour les agriculteurs :

A travers ce système, le propriétaire du champ de végétation doit pouvoir :

- Localiser ses champs sur une carte
- Suivre en temps réel les données météorologiques relatives à sa récolte.
- Visualiser la position des incendies éventuels et leur propagation.
- Recevoir une alerte de feu déclenché ou de risque majeur de feu

Pour l'agent ONPC :

Le système devrait permettre à un agent de l'ONPC de :

- Avoir un état de lieux global sur les incendies sur toute la région.
- Indiquer les positions des différents incendies et leur propagation.
- Collecter les données concernant les différentes récoltes et créer une base de données sur l'état des différents champs au niveau de sa région.
- Recevoir une alerte de feu déclenché ou de risque majeur de feu
- Collecter les données relatives aux différents incendies : position, propagation, et intensité afin de pouvoir mieux les contrôler et mettre en place les mesures nécessaires pour les éviter.

2.1.3 Besoins non fonctionnels

En terme de besoins non fonctionnels, notre système doit vérifier certains critères afin d'assurer sa qualité en tant que logiciel. Parmi ces besoins, nous citons principalement :

- **Précision** : Le modèle à développer doit fournir des résultats précis étant donné la sensibilité des données à traiter et des événements à surveiller.
- **Rapidité** : Le fonctionnement du logiciel doit être assez rapide afin de satisfaire l'objectif principal du système qui est l'intervention rapide dans le cas d'incendie.
- **Ergonomie** : Le logiciel doit être facile à comprendre et à utiliser afin de permettre la réduction de la période d'adaptation des nouveaux utilisateurs.

- **Fiabilité** : Le logiciel doit être fiable et assurer des résultats de détection et/ou de prédiction correctes afin d'éviter toute perte de ressources.
- **Accessibilité et disponibilité** : Le système doit être capable de fonctionner sans arrêt afin de pouvoir informer les différents acteurs à tout moment de tout incendies.

2.1.4 Besoins du domaine

Les besoins du domaine représentent les requis qui proviennent du domaine d'application du système et qui reflètent les caractéristiques de ce domaine. Ils sont principalement :

- **Données météorologiques en temps réel** : Le système doit être capable de collecter et d'analyser des données météorologiques en temps réel, telles que la température, l'humidité, la quantité de précipitations, la vitesse et la direction du vent, qui sont des facteurs importants dans la prévision des incendies.
- **Estimation des risques** : Le système doit être capable d'analyser les données recueillies et d'évaluer les risques potentiels d'incendie dans différentes régions, en tenant compte des conditions météorologiques, topographiques et des activités humaines.
- **Système d'alerte** : Le système proposé doit fournir des alertes rapides et précises en cas de détection d'incendie, permettant ainsi une intervention rapide des autorités concernées.
- **Géolocalisation** : Le système doit être capable de localiser précisément les incendies détectés, permettant ainsi aux autorités d'intervenir rapidement et efficacement.

2.2 Modélisation des besoins

Dans cette partie, nous modélisons notre système par un ensemble de diagrammes qui vont mieux visualiser les relations entre les différents composants et les différents scénarios d'utilisation.

2.2.1 Diagramme de cas d'utilisation général

Ce diagramme permet de visualiser les principales interactions entre les acteurs déjà identifiés et le système sous forme de cas d'utilisation.

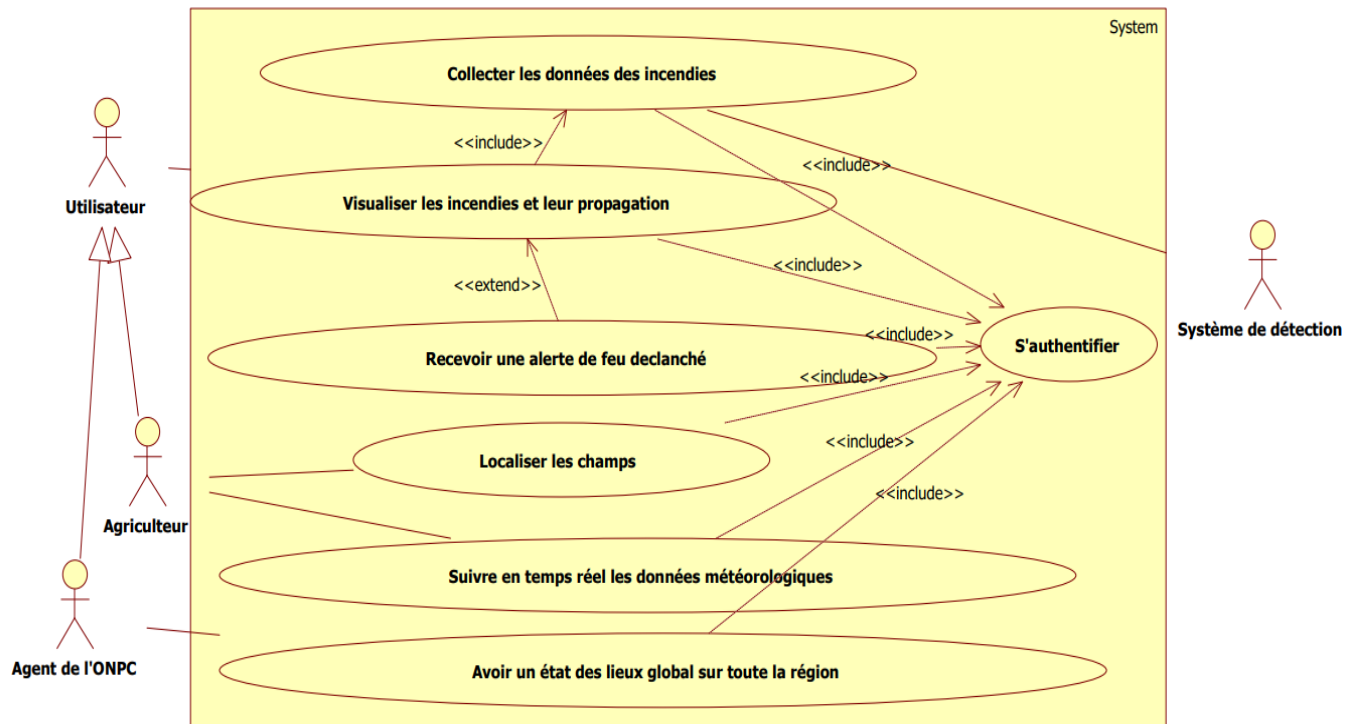


FIGURE 2.1 – Diagramme des cas d'utilisation général

Comme indiqué sur la figure 2.1, l'agriculteur et l'ONPC partagent des fonctionnalités communes relatives à la visualisation des données météorologiques, le suivi du risque de feu et la notification en cas d'alerte. Cependant, l'agriculteur a une visibilité limitée à ses champs inscrits sur le système, tandis que l'ONPC possède une vision étendue sur l'ensemble de la région ce qui lui permet d'avoir accès à un état de lieux et historique global. Cette différenciation se fait au niveau de l'authentification qui consiste en une étape primordiale à tous les acteurs. D'autre part, la visualisation des données météorologiques et le suivi de la propagation des feux éventuels nécessitent la présence d'un système de détection fiable et disponible.

2.2.2 Modélisation détaillée

Dans cette partie, nous présentons les cas d'utilisation de chaque acteur séparément afin de mieux cerner leurs spécificités et cas particuliers.

2.2.2.1 Diagrammes spécifiques à l'agriculteur

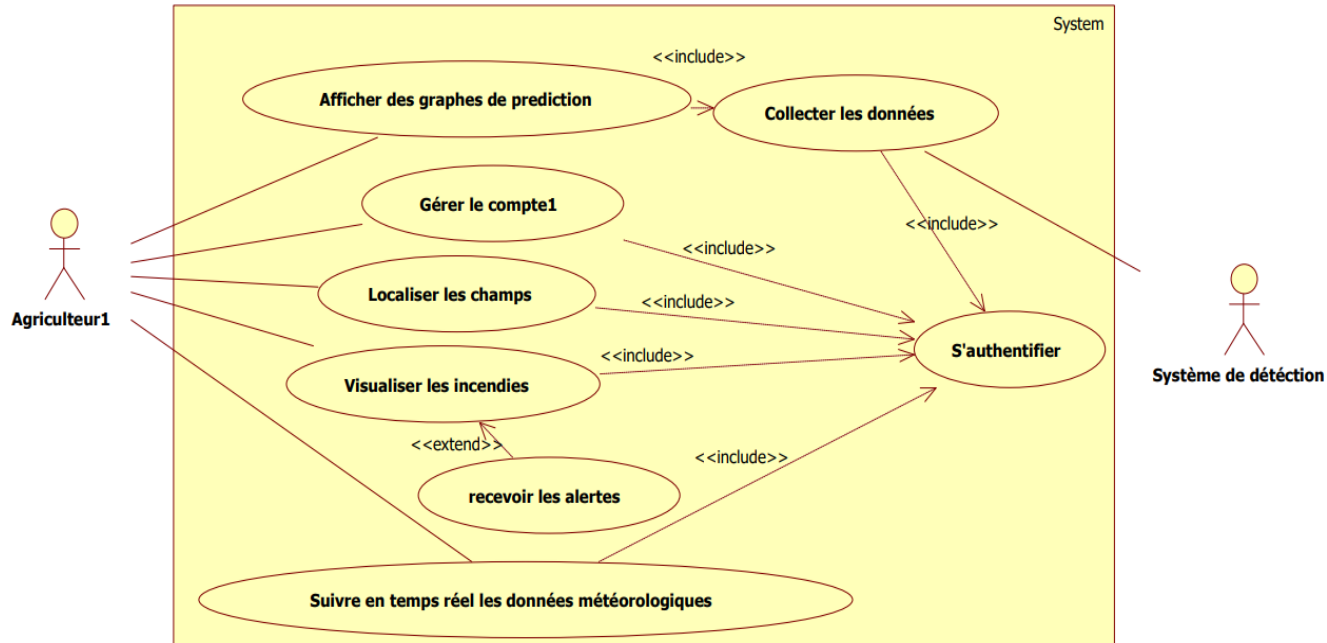


FIGURE 2.2 – Diagramme des cas d'utilisation associé à l'agriculteur

Les différents cas d'utilisation spécifiques à l'agriculteur dans notre figure 2.2 sont :

- **Afficher les graphes de prédiction** : Dans l'espace système de l'agriculteur, il a accès à un diagramme qui va lui permettre de cerner et de suivre l'évolution de la valeur de certaines grandeurs caractéristiques de ses champs de végétation comme le NDVI qui peut aider l'agriculteur à prédire les incendies. Dans le cas d'une erreur d'importation de données, nous prévoyons la possibilité de charger les données de la dernière session (son dernier accès au système) ce qui lui permet de garder un état fonctionnel. L'interaction entre l'agriculteur et le système est représentée à l'aide du diagramme de séquences de la figure 2.3.

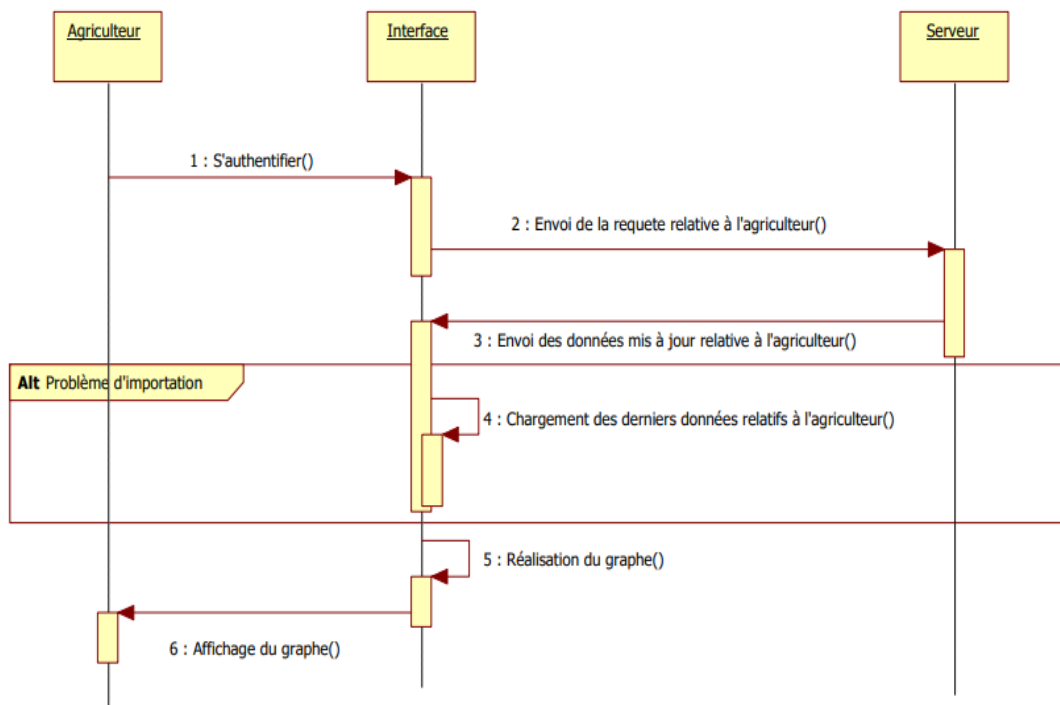


FIGURE 2.3 – Diagramme de séquences relatif au cas d'utilisation "Afficher les graphes de prédiction"

- **Gérer le compte** : L'agriculteur, afin de pouvoir utiliser les différentes fonctionnalités du système, doit obligatoirement créer un compte qui va contenir ces données personnelles avec des données relatives à un champ qui lui appartient et qui va être considéré comme point de référence ou champ par défaut. Le système offre à l'agriculteur la possibilité de modifier ces données à tout moment comme illustré par le diagramme 2.4.

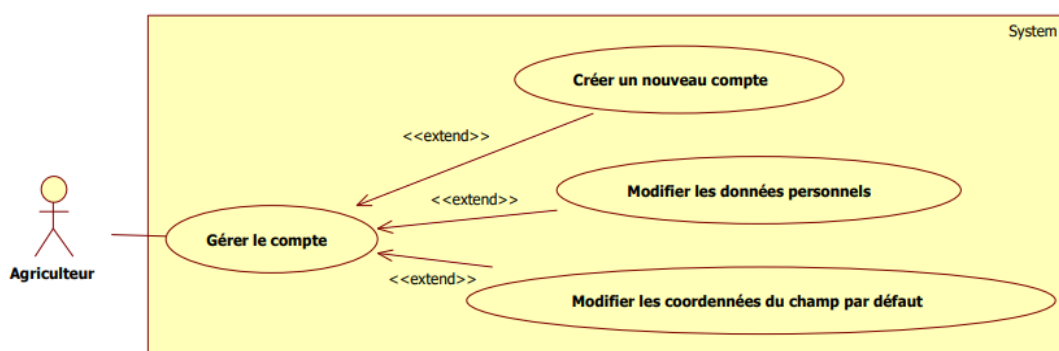


FIGURE 2.4 – Diagramme des cas d'utilisation spécifique au cas d'utilisation "gérer le compte"

- **Suivre en temps réel les données météorologiques** : L'interface permet à l'utilisateur de visualiser les différentes données météorologiques (température, précipitation, humidité, vent, etc.) relatives à un emplacement qu'il indique ou à un emplacement par

défaut. Le diagramme de séquence 2.5 décrit les différents scénarios possibles pour ce cas d'utilisation.

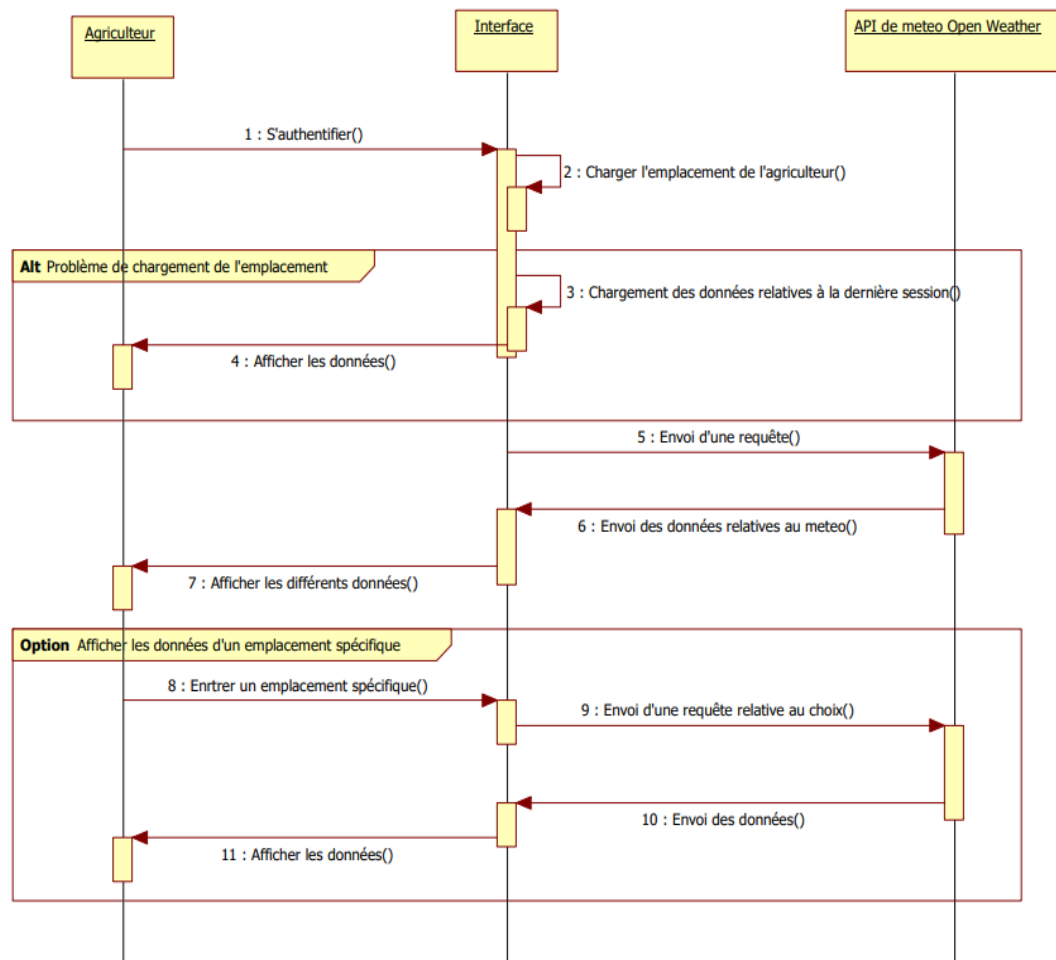


FIGURE 2.5 – Diagramme de séquences relatif au cas d'utilisation "gérer le compte"

- **Localiser les incendies** : Le but principal de notre système est de localiser les incendies sur une carte géographique à l'aide d'un modèle qui prend en entrée les données en temps réel des champs stockées sur un serveur et permet de calculer les positions des incendies. Cela est décrit par le diagrammes de séquences de la figure 2.6.

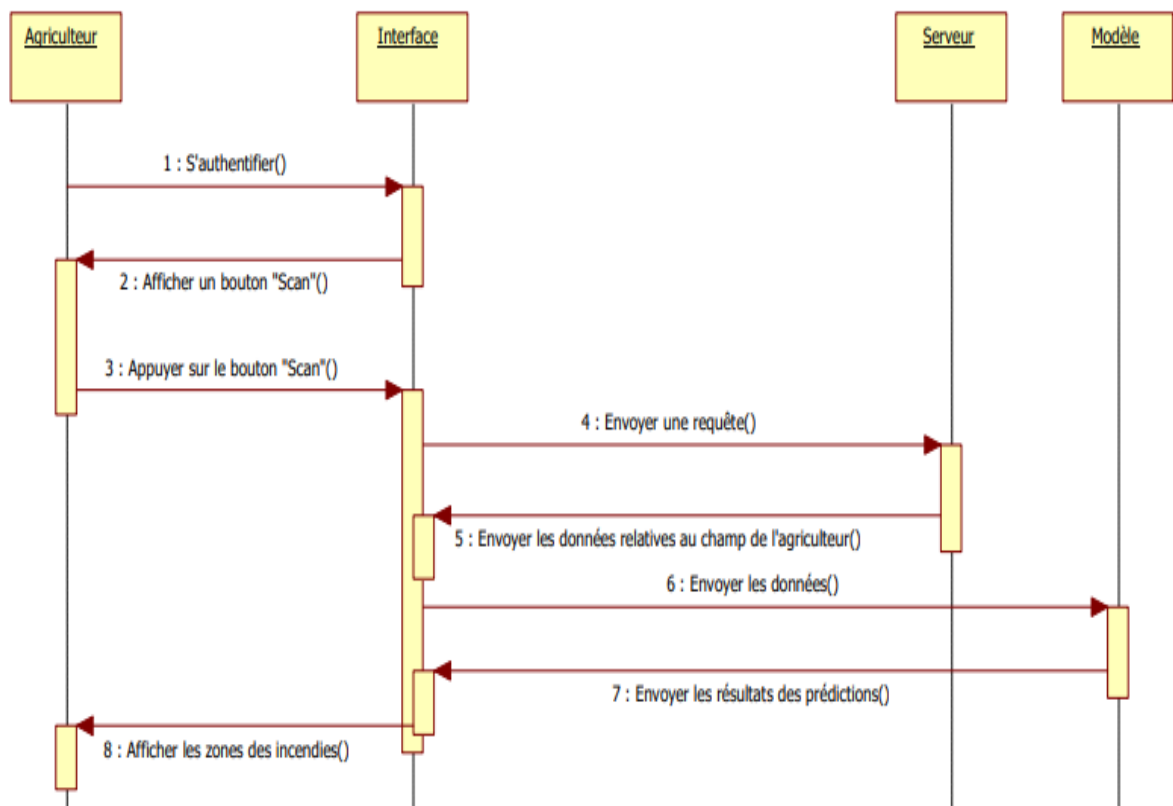


FIGURE 2.6 – Diagramme de séquences relatif au cas d'utilisation "Visualiser les incendies"

2.2.2.2 Diagramme spécifiques à l'agent de l'ONPC

Dans ce diagramme d'utilisation, il existe quelques cas d'utilisation qui ont déjà été traité dans l'analyse des besoins de l'agriculteur vue que les deux héritent d'une même classe "utilisateur". Nous nous intéressons sur les cas d'utilisation spécifiques à l'agent de l'ONPC, à savoir :

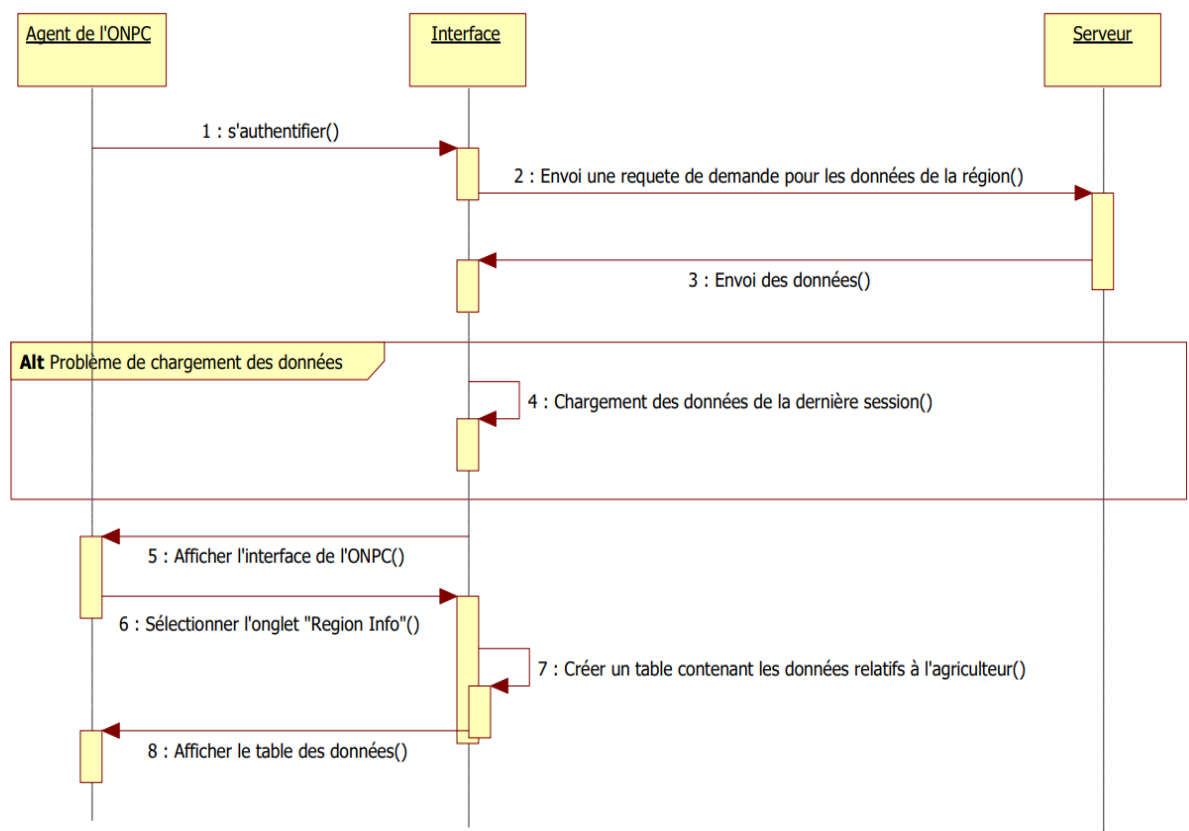


FIGURE 2.8 – Diagramme de séquences relatif au cas d'utilisation "Afficher les données des agriculteurs"

- **Localiser les agriculteurs et leurs champs** : L'agent de l'ONPC peut, grâce à un système de géolocalisation, localiser les différents agriculteurs sur une carte et savoir leurs positions et la position de leurs champs. Il peut savoir en plus quel agriculteur a été affecté par un feu de végétation dernièrement. La figure 2.9 discute et détaille ce scénario.

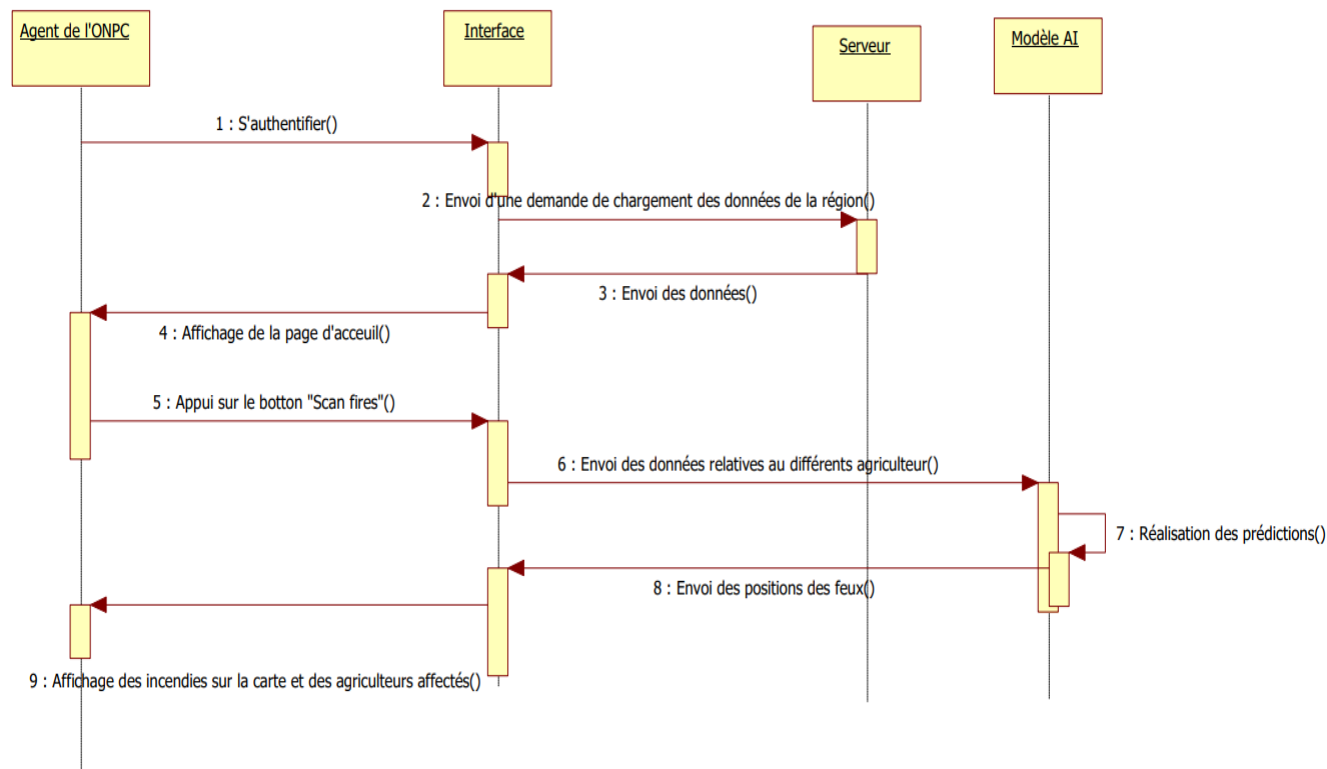


FIGURE 2.9 – Diagramme de séquences relatif au cas d'utilisation "Localiser les agriculteurs et leurs champs"

Conclusion

L'analyse des besoins procure une vision plus claire du sujet et une compréhension plus profonde des tâches à réaliser. Elle mène également à prévoir les problèmes à rencontrer et chercher les solutions permettant de les contourner. Par conséquent, nous devons spécifier et chercher les moyens possibles pour mettre en place ces solutions afin de réaliser une bonne conception du travail. C'est pourquoi nous essayons tout au long de ce chapitre de bien présenter les besoins fonctionnels et non fonctionnels du sujet. Ces besoins vont être la base sur laquelle nous réalisons la conception de notre système qui fera l'objet du chapitre suivant.

Chapitre 3

Conception

Dans ce chapitre, nous exposons les étapes de la conception de notre système. Cette partie est essentielle à la préparation de notre projet et devrait permettre une réalisation rapide et efficace. En effet, les phases d'analyse et d'étude de l'existant ont abouti à une description complète des besoins du système ce qui nous incite à réfléchir à l'architecture générale de l'application à réaliser.

Etant donné que notre travail consiste en deux parties distinctes à savoir : Développer un modèle de prédiction des feux dans les champs de végétation et développer une application de type *dashboard* permettant l'interaction avec l'utilisateur, ce chapitre comprendra, donc, une conception logicielle de l'application, et une conception de données relative du *dataset* utilisé.

3.1 Description et Conception des données

3.1.1 Description du jeu de données

Une recherche récente [5] montre que les feux de végétation sont contrôlés par les interactions entre trois facteurs : le climat, la végétation et les actions humaines. En se basant sur ce constat, nous orientons notre choix de données vers les jeux contenant à la fois des informations sur les conditions climatiques et les caractéristiques de végétation. Une recherche minutieuse a abouti à la récupération d'un jeu de données créé par télédétection et contenant des données liées à l'état des cultures (NDVI : Normalized Difference Vegetation Index), aux conditions météorologiques (LST : Land Surface Temperature) ainsi que l'indicateur d'incendie « Anomalies thermiques » [6]. Les trois paramètres ont été collectés à partir du MODIS (Moderate Resolution Imaging Spectroradiometer), un instrument embarqué à bord de la plateforme Terra et téléchargées à partir du site officiel du Land Processes Distributed Active Archive Center (LP DAAC) de la NASA. Les données collectées ont subi plusieurs techniques de prétraitement avant de créer l'ensemble de données final. La zone d'étude est composée de plusieurs zones situées au centre du Canada. La superficie de cette zone est d'environ 2 millions d'hectares. Ces zones diffèrent par leur taille, leur période de brûlage, la date du brûlage et leur étendue. Le Dataset est composé de quatre colonnes, les trois premières colonnes sont NDVI, LST et Thermal Anomalies et la quatrième colonne représente la classe

correspondante (fire ou no_fire), comme décrit ci-dessous.

- **Variable Cible** : Une variable cible étiquetée **fire** pour une zone susceptible d’être brûlée et **no fire** sinon.
- **NDVI (Indice de végétation par différence normalisée)** : Cet indice calcule la verdure et la densité de la végétation présente dans une image. Il est calculé à partir des mesures des facteurs de réflexion des canaux rouge (R) et proche infrarouge (PIR) mesurées dans la bande visible par des capteurs embarqués sur drones ou satellites. La valeur de cet indice varie entre -1 et 1.
- **LST (La température de la surface terrestre)** : Cette variable indique la température de l’atmosphère juste au dessus du sol en tenant en compte plusieurs processus tel que les échanges d’énergie, d’eau et l’influence de la végétation.
- **BURNED AREA** : C’est un indicateur d’anomalie thermique pour évaluer les zones brûlées pendant les saisons d’incendie.

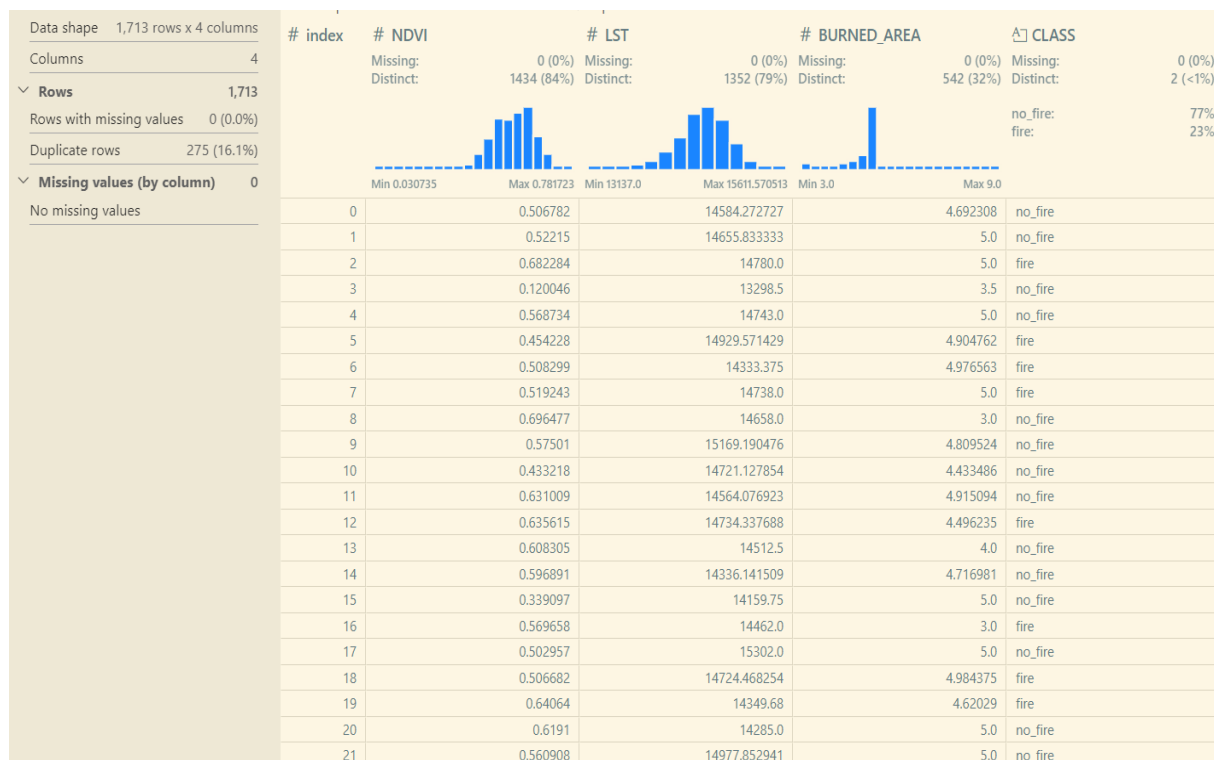


FIGURE 3.1 – Extrait du jeu de données

```
features = list(data.select_dtypes(include = np.number).columns)
data=data.drop(columns="CLASS")
```

```
['NDVI', 'LST', 'BURNED_AREA']
```

Listing 1 – Extraction de la cible et des attributs

Ce jeu de données contient 1713 lignes : 1327 instances de classe "no fire" et 386 instances de classe "fire".

```
target.value_counts()
```

```
CLASS
no_fire    1327
fire        386
Name: count, dtype: int64
```

Listing 2 – Distribution des données selon leurs classes

Notre objectif est de développer un modèle de classification capable de mettre en évidence la probabilité d'avoir un feu de végétation en s'appuyant sur les paramètres décrits précédemment. Pour ce faire, nous détaillons ici les étapes de nettoyage et pré-traitement menées sur l'ensemble de données tout en justifiant nos choix à chaque étape du processus.

3.1.2 Pré-traitement et préparation des données :

La préparation des données joue un rôle central dans l'exactitude et la fiabilité de l'analyse du modèle et permet d'en tirer des informations significatives et ainsi des décisions éclairées. Ce processus de pré-traitement fait référence à la manipulation, le filtrage et l'augmentation des données avant qu'elles soient traitées.

Les principales étapes sont :

- Normalisation et Encodage des caractéristiques catégoriques
- Gestion des valeurs aberrantes
- Mise à l'échelle (Scaling)
- Gestion des données manquantes
- Nettoyage des données pour plus de précision

3.1.2.1 Normalisation et Encodage des caractéristiques catégoriques

L'encodage des caractéristiques catégoriques est la représentation des caractéristiques données sous forme de valeurs discrètes par des codes entiers, cet encodage peut se traduire par une amélioration des performances du modèle d'apprentissage lorsque nous disposons de données ordinales ou nominales susceptibles de fournir une valeur. Il existe plusieurs techniques d'encodage dont les plus couramment utilisés sont :

- **Ordinal Encoding** : cette méthode consiste à attribuer une valeur entière unique à chaque catégorie suivant un ordre hiérarchisé, elle est généralement utilisée pour l'encodage des attributs. Elle peut être appliquée sur une entrée multidimensionnelle.
- **Label Encoding** : cette méthode consiste à attribuer une valeur entière à chaque catégorie sans avoir un ordre hiérarchisé. Elle est utilisée pour l'encodage de la classe cible pour un problème de classification.
- **One Hot Encoding** : cette méthode représente chaque catégorie par un vecteur binaire dont une seule valeur est 1 (présentant la catégorie) et les autres sont mises à 0.
- **Frequency Encoding** : cette méthode consiste à attribuer à chaque catégorie sa fréquence d'apparition de cette catégorie dans la colonne, elle est utile lorsque nous voulons tenir compte de la distribution de chaque catégorie.
- **Target Encoding** : cette méthode consiste à remplacer chaque catégorie par la moyenne de la variable cible pour cette catégorie (taux de conversion).

Pour notre travail, nous utilisons l'**encodage par étiquette** provenant de **sklearn.preprocessing** où nous attribuons la valeur **0** pour la classe «no fire» et la valeur **1** pour la classe «fire».

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
target_mapping={'no_fire':0, 'fire':1}
target= encoder.fit_transform(target.map(target_mapping))
target=pd.DataFrame(target)
```

Listing 3 – Encodage de la variable cible

	NDVI	LST	BURNED_AREA	CLASS
0	0.506782	14584.272727	4.692308	0
1	0.522150	14655.833333	5.000000	0
2	0.682284	14780.000000	5.000000	1
3	0.120046	13298.500000	3.500000	0
4	0.568734	14743.000000	5.000000	0
...	...	...	...	...
1708	0.621721	14406.000000	5.000000	0
1709	0.709715	14330.400000	4.851852	0
1710	0.613000	14751.500000	3.000000	1
1711	0.614471	13611.000000	4.921053	0
1712	0.549497	14269.777778	5.000000	1
1713 rows × 4 columns				

FIGURE 3.2 – Extrait du jeu de données après l'encodage

3.1.2.2 Mise à l'échelle des données (Data Scaling)

Il s'agit d'un processus couramment utilisé pour mettre à l'échelle des données dans un même intervalle. La normalisation ou la mise à l'échelle des données garantit que les variables sont comparables sur la même échelle, empêchant ainsi une variable de dominer la corrélation en raison de sa plus grande ampleur. En effet, la présence de valeurs très grandes ou très petites par rapport à d'autres peut poser des problèmes pour certaines techniques statistiques ou algorithmes d'apprentissage automatique qui se base sur la distance entre les observations tel que **KNN** et **SVM**. Plusieurs techniques de mise en échelle existent, telles que :

- **MinMax Scaler** : cette méthode consiste à transformer les valeurs afin qu'elles se trouvent dans une plage spécifique, généralement entre 0 et 1.
- **Standard Scaler** : cette méthode consiste à transformer les valeurs afin qu'elles aient une distribution de moyenne nulle et d'écart type égal à 1.

Pour notre cas, nous utilisons la technique de normalisation en important **MinMax Scaler** provenant de **sklearn.preprocessing**.

```

from sklearn.preprocessing import MinMaxScaler
scaler= MinMaxScaler()
scaler.fit(data)
scaler= MinMaxScaler().set_output(transform="pandas")
data=scaler.fit_transform(data)
data
datahis=data.hist()

```

Listing 4 – Normalisation

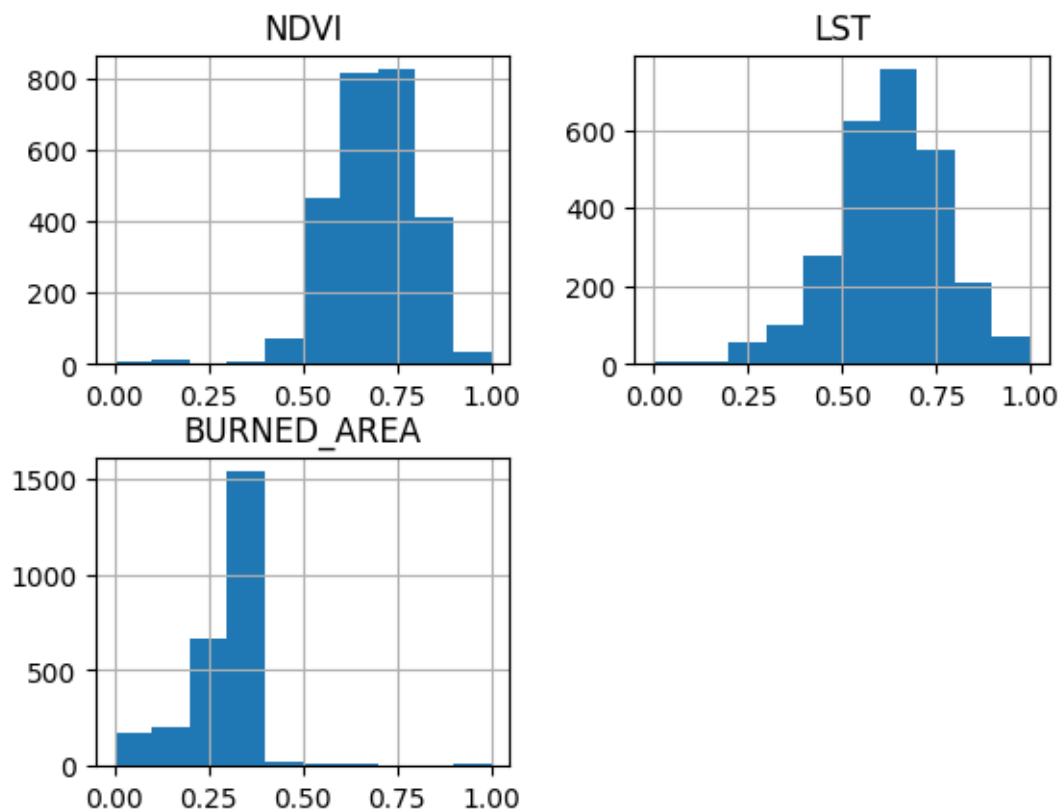


FIGURE 3.3 – Résultat de mise en échelle sur les 3 attributs du jeu de données

3.1.2.3 Nettoyage des données

Ce processus permet de détecter et supprimer les erreurs, les incohérences, les valeurs aberrantes, les duplicatas, les valeurs manquantes et autres problèmes afin d'assurer la cohérence et la fiabilité des données. Ceci aide à éviter les problèmes de sur-ajustement (over-fitting), sous-ajustement (under-fitting), le biais et la variance lors du fonctionnement du modèle d'apprentissage.

3.1.2.4 Détection et suppression des duplicatas et des valeurs manquantes

```
data.drop_duplicates()
data.dropna()
```

Listing 5 – Détection et suppression des duplicatas et des valeurs manquantes

3.1.2.5 Détection et suppression des valeurs aberrantes

Une valeur aberrante est une valeur extrêmement différente et éloignée de la plage où se localisent la plupart des valeurs de la même variable.

Les algorithmes d'apprentissage automatique sont sensibles à la distribution des données de telle sorte que la présence des valeurs aberrantes peut rallonger la phase d'entraînement et biaiser le modèle. La détection des valeurs aberrantes se fait à travers le tracé de boîte (**Box Plot**) qui est une représentation graphique des données montrant la médiane, les quartiles et les valeurs aberrantes qui sont représentées par toute valeur supérieure ou inférieure à 1.5 fois l'inter-quartile comme illustré sur la figure 3.4.

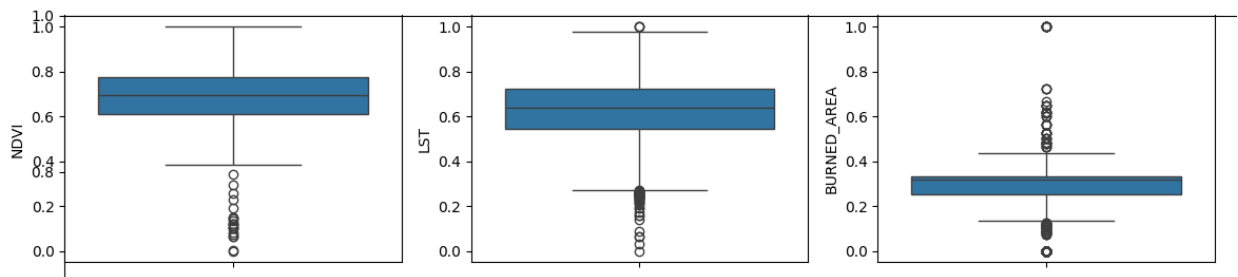


FIGURE 3.4 – Détection des valeurs aberrantes à travers le tracé de boîte.

```

Def remove_outliers(data, columns_with_outliers, target):
    for column in columns_with_outliers:
        lower_limit = data[column].mean() - 2 * data[column].std()
        higher_limit = data[column].mean() + 2 * data[column].std()
        outliers_index = data[(data[column] < lower_limit) |

        (data[column] > higher_limit)].index
        data = data.drop(outliers_index)
        target = target.drop(outliers_index)
    return data, target

data, target=remove_outliers(data,["NDVI", "LST", "BURNED_AREA"],target)

```

Listing 6 – Détection et suppression des valeurs aberrantes

La figure 3.5 montre le résultat du nettoyage et la suppression des valeurs incohérentes détectées sur les 3 attributs de notre dataset.

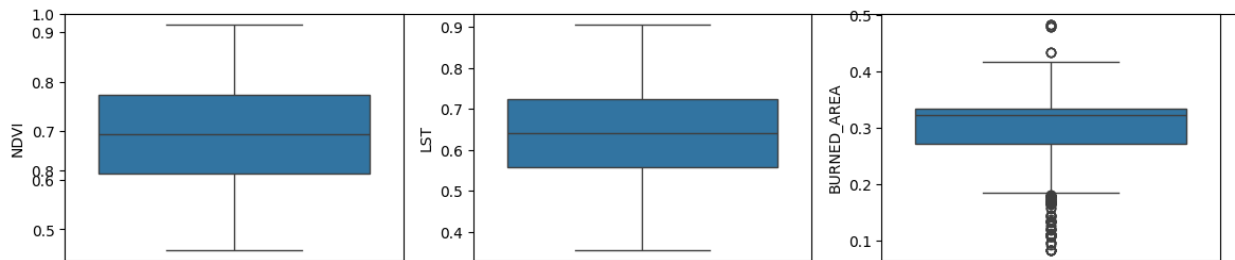


FIGURE 3.5 – Résultat du nettoyage des valeurs aberrantes

3.1.3 Entraînement et Évaluation des modèles

L'entraînement du modèle d'apprentissage consiste à présenter à l'algorithme un jeu de données afin d'établir des relations entre les attributs (features) et les étiquettes (labels), cette phase est cruciale car elle permet au modèle d'ajuster ses hyper-paramètres pour minimiser l'écart entre les prédictions et les étiquettes réelles.

Cette phase comprend les étapes suivantes :

1. **Data Preparation** : ça consiste à diviser nos données en ensembles d'entraînement pour entraîner le modèle et de test pour évaluer ses performances.
2. **Choix du modèle**
3. **Entraînement du modèle**
4. **Évaluation du modèle** : ça consiste à évaluer les performances du modèle.
5. **Optimisation et amélioration des performances du modèle** : ceci se fait en ajustant les hyperparamètres, en calibrant les prédictions du modèle ou en utilisant les mathodes à base d'ensemble telles que *Bagging*, *Boosting* et *Stacking*.

3.1.3.1 Entraînement du modèle

Pour tous les modèles étudiés, nous appliquons le principe de **Validation croisée** lors de la phase d'entraînement, cette méthode consiste à diviser le jeu de données en **K** ensembles, le modèle sera entraîné **K** fois en utilisant à chaque fois **K-1** ensembles et un ensemble de validation (test). Les performances du modèle sur chaque ensemble serviront pour l'obtention d'une mesure globale de la performance. Ce principe nous permet d'éviter le problème de sur-apprentissage et mettre en évidence la capacité de généralisation du modèle . Pour ceci, nous concevons la fonction **crossvalidate** comme suit :

```
from sklearn.model_selection import cross_val_score
def crossvalidate(model):
    cv = ShuffleSplit(n_splits=10, test_size=0.3, random_state=0)
    cv_scores = cross_val_score(model, data, target, cv=cv)
    CV=cv_scores.mean()
    return CV
```

Listing 7 – Validation croisée d'un modèle donné

3.1.3.2 Évaluation du modèle

L'évaluation du modèle se fait à travers plusieurs outils et en examinant des métriques spécifiques à savoir :

- **L'exactitude (*Accuracy*)** : elle mesure la capacité d'un modèle à créer des prédictions correctes et exactes.
- **La précision (*Precision*)** : cette métrique évalue la qualité des prédictions positives du modèle, c-à-d, les instances réellement positives parmi les instances prédites positives.
- **Le Rappel (*Recall*)** : cette métrique évalue la capacité du modèle à capturer des instances positives parmi les instances réellement positives.
- **Le score F1** : c'est une mesure décrivant la capacité d'un modèle à classer les instances positives tout en minimisant les fausses prédictions. Elle combine la précision et le rappel.

```

from sklearn.model_selection import cross_val_predict
from sklearn.metrics import accuracy_score, precision_score,
recall_score, roc_auc_score, mean_absolute_error

def get_the_score_of_the_model(model):
    predictions = cross_val_predict(model, data, target, cv=5)

    accuracy = accuracy_score(target, predictions)
    train_error = mean_absolute_error(target, model.predict(data))
    test_error = mean_absolute_error(target, predictions)
    print("Train error:", train_error)
    print("Test error:", test_error)
    print("Accuracy:", accuracy)

    precision = precision_score(target, predictions)
    recall = recall_score(target, predictions)
    print("Precision:", precision)
    print("Recall:", recall)

    auc = roc_auc_score(target, predictions)
    print("AUC:", auc)

```

Le calcul de ces métriques se base sur une **matrice de confusion** qui est considérée comme un outil permettant l'évaluation de la performance du modèle d'apprentissage automatique sur un ensemble de données de test en comparant ses prédictions par rapport aux valeurs réelles. Cette matrice comporte quatre catégories :

- **TRUE POSITIVE** :valeurs prédites correctement prédites comme étant réellement positives.
- **FALSE POSITIVE** :valeurs réellement négatives mais prédites comme étant positives.
- **TRUE NEGATIVE** :valeurs prédites correctement prédites comme étant réellement négatives.
- **FALSE NEGATIVE** :valeurs réellement positives mais prédites comme étant négatives.

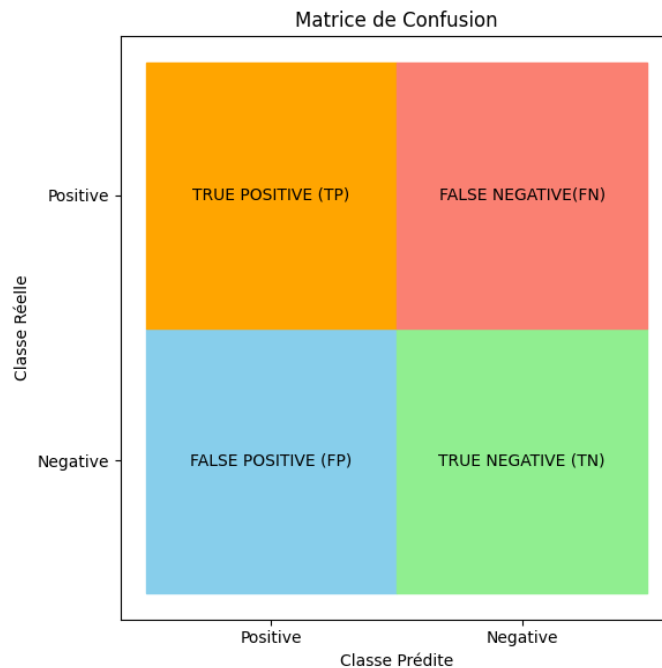


FIGURE 3.6 – Matrice de Confusion

Cet outil nous permet de mettre en évidence les métriques identifiées précédemment de la manière suivante :

$$\textbf{Exactitude} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.1)$$

$$\textbf{Precision} = \frac{TP}{TP + FP} \quad (3.2)$$

$$\textbf{Recall} = \frac{TP}{TP + FN} \quad (3.3)$$

$$\textbf{Score F1} = \frac{2 \cdot \textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}} \quad (3.4)$$

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix

def get_the_confusion_matrix_of(model):
    predictions = cross_val_predict(model, data, target, cv=5)
    matricedeconfusion=Confu
    (target,predictions)
    return cm
```

Listing 8 – Matrice de confusion

Pour affiner encore l'évaluation de chaque modèle donné, nous concevons les méthodes nécessaires permettant le traçage des courbes *ROC* et *Precision-Recall*.

- **Courbe Precision-Recall** : c'est une courbe représentant la variation de la valeur de la précision en fonction de la variation de la valeur du rappel. Il est décelable à partir de l'AUC (Area Under Curve).
- **Courbe ROC** : cette courbe est la représentation graphique de la variation de la valeur de la sensibilité (taux des TRUE POSITIVES) en fonction de celle de la spécificité (taux des FALSE POSITIVES) sur différents seuils de classification. Ceci met en valeur la capacité d'un modèle à faire la distinction entre les classes.

```

from sklearn.model_selection import cross_val_predict
from sklearn.metrics import PrecisionRecallDisplay, RocCurveDisplay
import matplotlib.pyplot as plt
from sklearn.dummy import DummyClassifier

dummyclassifier = DummyClassifier(strategy="stratified")

def get_the_PRCURVE_and_the_ROC(model):
    predictions = cross_val_predict(model, data,

    target, cv=5)
    predictions_dummy = cross_val_predict(dummyclassifier, data,

    target, cv=5)

    fig, axs = plt.subplots(ncols=2, nrows=1, figsize=(15, 7))

    PrecisionRecallDisplay.from_predictions(target, predictions,

    ax=axs[0], name="Model")
    PrecisionRecallDisplay.from_predictions(target,

    predictions_dummy, ax=axs[0], name="Dummy", linestyle="--",

    color="tab:green")
    axs[0].set_xlabel("Recall")
    axs[0].set_ylabel("Precision")
    axs[0].set_xlim(0, 1)
    axs[0].set_ylim(0, 1)
    axs[0].legend(loc="lower left")
    axs[0].set_title("Precision-recall curve")

    RocCurveDisplay.from_predictions(target, predictions,

    ax=axs[1], name="Model")
    RocCurveDisplay.from_predictions(target, predictions_dummy,

    ax=axs[1], name="Dummy", linestyle="--", color="tab:green")
    axs[1].set_xlabel("False positive rate")
    axs[1].set_ylabel("True positive rate")
    axs[1].set_xlim(0, 1)
    axs[1].set_ylim(0, 1)
    axs[1].legend(loc="lower right")
    axs[1].set_title("ROC")

    plt.tight_layout()
    plt.show()

```

3.1.3.3 Optimisation et Amélioration des performances :

Calibrage des hyper-paramètres : Cette phase consiste à effectuer une recherche afin de trouver la meilleure combinaison des valeurs des hyper-paramètres propres à un modèle lui permettant d'avoir de meilleures performances.

Plusieurs méthodes peuvent être utilisées pour faire ce calibrage et peuvent être coûteuses en terme de temps pour une grande plage d'hyper-paramètres ou carrément inefficaces pour une petite plage d'hyper-paramètres.

Pour notre travail, une recherche minutieuse a aboutit à l'adoption de l'outil **Optuna** afin d'effectuer cette tâche. Il s'agit d'un outil de recherche automatisée permettant de trouver les hyper-paramètres optimaux d'un modèle d'apprentissage automatique. Optuna offre trois fonctionnalités :

- **Espaces de recherche enthousiastes**
- **Algorithmes de pointe :** effectuer des recherches sur des grandes espaces et élaguer des essais sans compromis.
- **Parallélisation facile :** paralléliser la recherche d'hyperparamètres sur plusieurs threads ou processus sans modifier le code.

```
import optuna

def objective(trial):
    # Définir la distribution des hyperparamètres (plage, taille de l'incrément , and logari
    param = trial.suggest_float(param_name, borne_inf, borne_sup, step=None, log=False)
    classifieur_obj = Model(param=param) # Define the model
    classifieur_obj.fit(data_train, target_train)
    accuracy = classifieur_obj.score(data_test, target_test)
    return accuracy

sampler = optuna.samplers.TPESampler(seed=42) # Define the sampler
study = optuna.create_study(sampler=sampler, direction='maximize')
study.optimize(objective, n_trials=80) # Define the number of trials

print("Best_trial:", study.best_trial.number)
print("Best_accuracy:", study.best_trial.value)
print("Best_hyperparameters:", study.best_params)
```

Listing 10 – Implémentation de l'Optuna

Étalonnage du modèle : Pour un modèle d'apprentissage automatique prédictif, il y a la nécessité de la prévision d'une étiquette pour une observation donnée, cela offre une certaine flexibilité dans la manière à interpréter les prédictions (fonction seuil et incertitude de la probabilité).

Certains modèles, et précisément linéaires car il n'établissent pas directement des probabilités, peuvent établir une distribution de probabilités qui n'est pas conforme à la distribution observée dans le jeu de données.

Notons que certains algorithmes sont ajustés de telle sorte qu'il soient calibrés dès le départ tel que **la régression logistique**

Il existe deux approches pour effectuer ceci :

- **Diagrammes de fiabilité** un tracé linéaire de ce qui était prédit par rapport à la fréquence de probabilité prédite. Ainsi, plus les points visualisés sont proches tout au long de la diagonale principale, meilleure est la prédiction.
- **Calibrage des probabilités** : peut se faire à travers l'approche de la mise en échelle à plat si l'allure de la courbe est sigmoïde ou l'approche de la régression isotonique pour une autre forme plus complexe.

Pour notre modèle, après avoir ajusté les hyper-paramètres de chaque modèle, nous faisons recours à l'étalonnage de notre modèle en traçant d'abord le diagramme de fiabilité et à partir de chaque allure de la courbe de fiabilité, nous effectuons cette tâche de calibrage de probabilités selon l'approche convenable amenant à une meilleure performance.

Chaque tracé représente la forme de la courbe d'étalonnage avant et après le calibrage.

```

import matplotlib.pyplot as plt
from sklearn.calibration import calibration_curve
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import accuracy_score
model.fit(data_train, target_train)
model_calibrated = CalibratedClassifierCV(model_1_tuned, cv=3,
method='sigmoid' ou bien 'isotonic')
train_the_model(model_calibrated)
get_the_score_of_the_model(model_calibrated)
prob = model.predict_proba(data_test)[: , 1]
prob1 = model_calibrated.predict_proba(data_test)[: , 1]
fraction_of_positives, mean_predicted_value
= calibration_curve(target_test, prob, n_bins=10)
fraction_of_positives1, mean_predicted_value1
= calibration_curve(target_test, prob1, n_bins=10)
plt.figure(figsize=(10, 8))
plt.plot(mean_predicted_value, fraction_of_positives, marker='o',
label='model', color='b')
plt.plot(mean_predicted_value1, fraction_of_positives1, color='r',
marker='o', label='calibrated model')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray',
label='Perfectly calibrated')
plt.xlabel('Mean predicted probability')
plt.ylabel('Fraction of positives')
plt.title('Calibration Curve')
plt.legend()
plt.show()

```

Listing 11 – Courbe d'étalonnage

Choix de la méthode d'ensemble : L'apprentissage d'ensemble est un paradigme dont lequel plusieurs modèles de base appelés **apprenants faibles** (weak learners), représentant des blocs de construction, sont entraînés sur le même jeu de données et combinés ensemble afin d'obtenir de meilleurs résultats. Ces modèles de base sont moins performants soit parce qu'ils ont un fort biais (modèles à faible degré de liberté) soit parce qu'ils ont une grande variance (modèles à haut degré de liberté). L'idée des méthodes d'ensemble est de réduire le biais et la variance afin d'aboutir à la construction d'un apprenant fort (strong learner). Notons que le choix de modèles de base doit être cohérent avec la façon dont nous agrégeons ces modèles, c-à-d, si nous choisissons des modèles de base à faible biais mais à forte variance, ceci doit être avec une méthode d'agrégation, et vice versa. On distingue, pour ce faire, trois méthodes :

- **Bagging** pour des modèles de base homogènes, ça consiste d'abord à réaliser un jeu de données bootstrap en créant un nouveau jeu de données à partir du jeu de données initial. La création des bootstraps se fait en suivant les processus décrit sur la figure 3.7 :

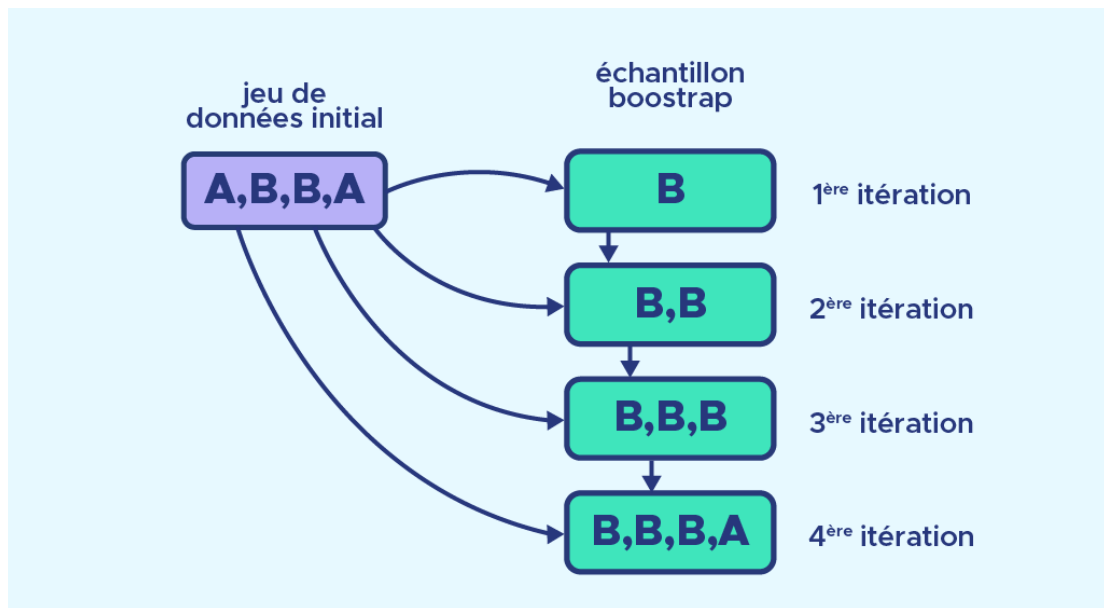


FIGURE 3.7 – Création d'un jeu de données *Bootstrap*

A chaque bootstrap, nous entraînons le modèle de base indépendamment et combiner les prédictions afin d'aboutir à une prédiction finale. Pour la classification, il s'agit d'un vote à la majorité ou une majorité pondérée. Le *bagging* consiste à obtenir un modèle d'ensemble avec moins de variance que les modèles base.

- **Boosting** dédié aux modèles de base homogènes. Il s'agit de combiner plusieurs apprenants faibles pour créer un apprenant fort. L'ensemble des modèles faibles est entraîné en série de sorte que chaque modèle suivant tente de corriger les erreurs du modèle précédent jusqu'à ce que l'ensemble du jeu de données d'apprentissage soit prédit correctement. Cette méthode consiste à créer un modèle d'ensemble moins biaisé que les modèles de base.
- **Stacking** utilisé avec des modèles de base hétérogènes. Cette méthode permet de créer un modèle moins biaisé même si la variance est réduite. Ce modèle est composé de deux niveaux : D'abord, on entraîne les modèles du premier niveau sur le jeu de données en utilisant le principe de la validation croisée afin d'obtenir des prédictions. Ces prédictions vont être utilisées comme étant caractéristiques pour le modèle de deuxième niveau qui donnera une prédiction finale.

Pour un problème de classification, le modèle de deuxième niveau est la régression logistique.

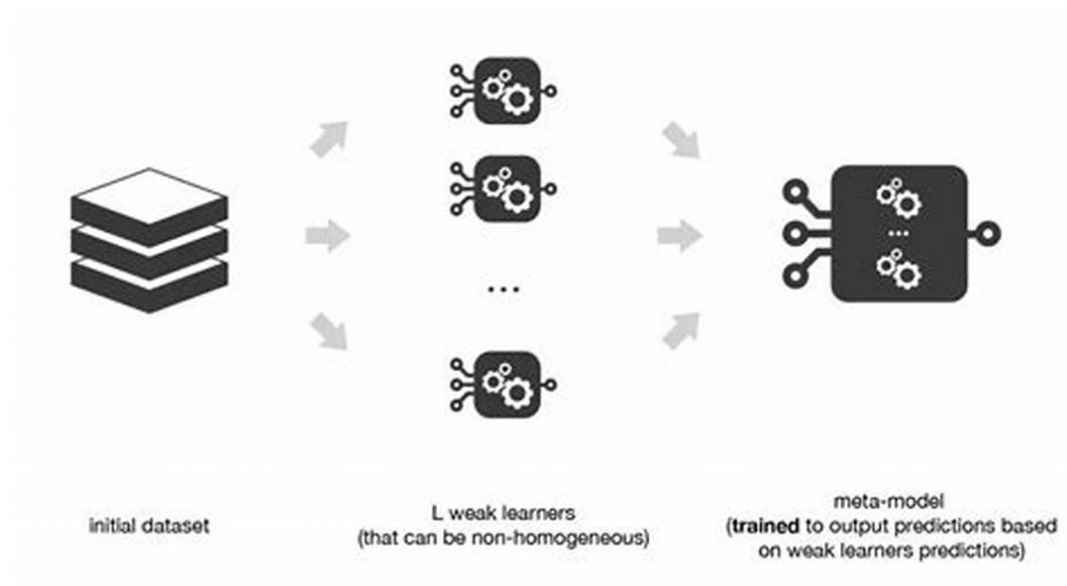


FIGURE 3.8 – *Stacking Model*

3.2 Conception logicielle

Après avoir procédé à la conception des données qui vont alimenter notre modèle d'apprentissage, nous présentons maintenant l'architecture logicielle de notre système afin de pouvoir y intégrer les données et le modèle d'apprentissage correctement et servir les besoins fonctionnels précédemment identifiés convenablement.

3.2.1 Choix de l'architecture logique

L'architecture logique est indépendante des considérations physiques hors de tout contexte d'exécution (machines, OS et réseaux,...). Dans cette partie, nous passons en revue quelques architectures logiques qui peuvent être utilisées et faire le choix qui s'aligne mieux avec nos besoins.

3.2.1.1 Architectures en flots de données (Dataflow)

Le style de flots de données est une architecture composée d'un ensemble de filtres qui sont inter connectées par des canaux de transfert. La figure 3.9 [URL10] décrit cette architecture avec les filtres représentés par l'ensemble des opérations formant un *pipeline*.



FIGURE 3.9 – Architecture en Flots des données [URL10]

Le **filtre** est ici un composant de transformation indépendant. Plus précisément, c'est un convertisseur qui attend des données d'entrée et produit des données de sortie dans des types prédéfinis. Le **Pipes ou canaux** est un ensemble de connecteurs qui relient les composantes source et récepteur et propagent les données. Il permet l'échange des données entre les filtres.

Avantages :

- ✦ Les filtres peuvent être conçus séparément qui va permettre de subdiviser tout problème en un ensemble de problèmes moins compliqués qui va permettre de faciliter leur conception et de les développer en parallèle ce qui réduit le temps de conception.
- ✦ Les filtres permettent aussi d'assurer une cohésion fonctionnelle forte avec un faible couplage tout en cachant leurs détails internes ce qui garantit l'abstraction du système. Ils peuvent être aussi réutilisés dans d'autres contextes.

Inconvénients :

- Les performances vont dépendre essentiellement des pipes utilisés
- Il est difficile d'implémenter cette architecture pour des systèmes interactifs (avec évènements)

3.2.1.2 Architectures centrées sur les données

Dans le style centré sur les données, un composant central comme un SGBD ou un Datawarehouse est responsable de la gestion des données (conservation, ajout, retrait, mise-à-jour, synchronisation, ..). La figure 3.10 [URL11] donne un exemple sur cette architecture.

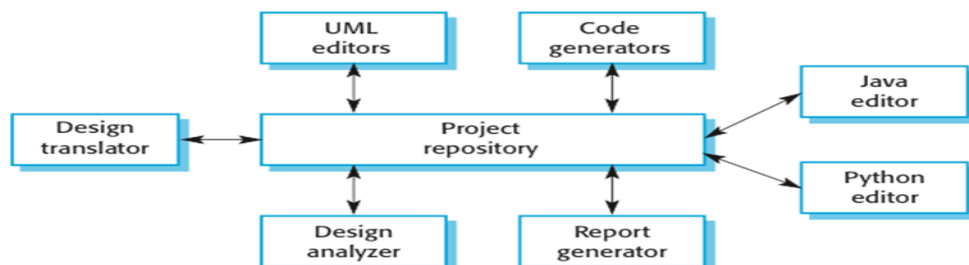


FIGURE 3.10 – Architecture centrée sur les données [URL11]

Les composants périphériques, nommées clients ou accesseurs, utilisent le composant central, nommé serveur de données (centre de données).

Ce centre de données peut être soit :

- **passif** : il se comporte comme une dépôt de données et se limite à la collection des données sans offrir aucune interaction avec les différents accesseurs.
- **actif** : il constitue alors une structure logique où une mémoire de travail appelée «tableau noir», accessible à plusieurs composants-clients, qui sert à communiquer des résultats intermédiaires et à coordonner ces transferts d'informations entre clients.

Avantages :

- + Ce style présente une méthode efficace pour partager de grand volume de données
- + Ce style est avantageux pour les applications impliquant des tâches complexes et nombreuses sur les données. Les sous systèmes n'ont pas à se préoccuper comment les données sont stockées et gérées.

Inconvénients :

- Le dépôt de données peut facilement constituer un goulot d'étranglement, tant du point de vue de sa performance que du chargement (trop d'accès simultanés)

3.2.1.3 Architectures à base d'appels et retours (Call & Return)

Le style appels et retours, aussi appelé "call and return" est simple et classique décomposant le système en une hiérarchie de contrôle. Ce style consiste à découper une fonctionnalité en sous-fonctionnalités qui sont également divisées en sous sous-fonctionnalités et ainsi de suite. Il existe plusieurs approches de ce style :

- **Programme principal et sous routines** : C'est une approche de contrôle centralisée avec un contrôle descendant (top-down) où les composantes sont l'ensemble des sous-routines connecté par l'intermédiaire des appels fonctionnels.
- **Orientée Objet** : Cette architecture se base sur la structuration d'un système en des modules faiblement couplés. Les composantes sont des paquets connectés par des interfaces.

Elle permet d'assurer

- + un faible couplage entre les objets
- + les objets représentent des entités du domaine d'application
- + une représentation des fonctionnalités verticales du système

Cependant, elle introduit un nombre d'inconvénients comme :

- les entités complexes qui sont difficile à représenter comme les objets
- les fonctionnalités transversales qui demandent un grand nombre de module

3.2.1.4 Synthèse et choix :

D'après cette analyse réalisée sur les différents styles architecturaux possibles, nous trouvons que le style en flots de données (dataflow) peut à un certain niveau servir notre objectif grâce à sa simplicité et sa nature linéaire. En plus, le style appels et retours semble aussi

pouvoir répondre à nos besoins vue que chaque fonction peut être séparée et divisée ce qui peut renforcer la maintenabilité de ce système et renforce son évolution.

Cependant, le style centré sur les données peut assurer les points suivants :

- ✦ Une meilleure répartition sécurisée des données, étant donné qu'elle est centralisée et l'accès est limité au membre ayant les privilèges nécessaires.
- ✦ Une amélioration au niveau des performances : La relation point à point permet de diminuer le temps d'exécution des différentes instructions.
- ✦ Facilité d'implémentation tout en assurant un coût réduit de maintenance et de mise à jour.

Pour toutes ces raisons, nous nous orientons vers une architecture logique centrée sur les données.

3.2.2 Choix de l'architecture physique

L'architecture physique va dépendre directement du contexte d'exécution d'où nous devons savoir faire un bon choix afin de pouvoir réaliser notre système et répondre aux différents besoins sans pour autant ajouter des difficultés supplémentaires. Il existe plusieurs approches que nous pouvons adopter parmi lesquelles nous nous limitons à l'étude des architectures 2-tiers et 3-tiers.

3.2.2.1 Architecture 2-tiers

L'architecture 2-tiers est une architecture qui vise à répartir un système informatisé sur des machines distantes, grâce à des matériels et des protocoles standards. Elle caractérise les systèmes clients/serveurs dans lesquels le client demande une ressource et le serveur la lui fournit directement. Elle se compose, comme le nom l'indique, de 2 parties :

- Un client ayant un poste qui envoie des requêtes
- Un serveur qui stocke les différentes données relatives au fonctionnement du système

La figure 3.11 [URL12] décrit en détail la relation entre les deux couches client/serveur.

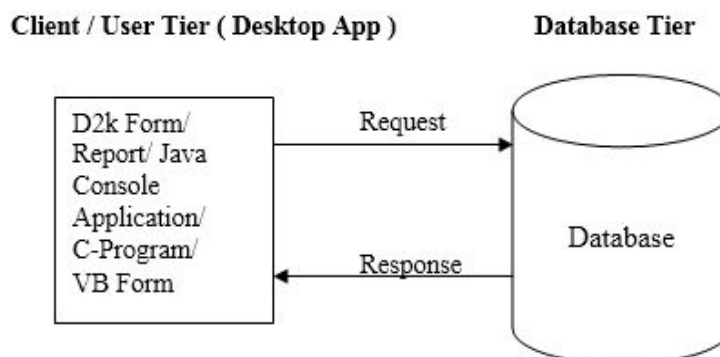


FIGURE 3.11 – Architecture 2-tiers [URL12]

Le client peut se charger d'une partie des traitements de données et le serveur s'occupe dans ce cas du stockage des données et du reste des traitements. Nous parlons ici d'un **client lourd**. Le serveur peut prendre les traitements dans leur totalité et le client se limite à l'accès et retrait des données. C'est plutôt un **client léger**. Finalement, un client lourd peut, en plus, se charger partiellement du stockage des données. Il s'agit d'un **client riche**.

Avantages :

- ✦ Elle permet d'assurer un couplage faible puisque le serveur n'a pas besoin des clients
- ✦ L'administration et la maintenance du serveur sont simplifiées.
- ✦ Cette architecture offre une souplesse au niveau de l'ajout et de la suppression des clients et assure l'intégrité des données vue sa structure centralisée.

Inconvénients :

- Les performances du système dépendent principalement du serveur.
- Le coût de mise en place du serveur est assez élevé vue le niveau de performance demandé.

3.2.2.2 Architecture 3-tiers

L'architecture 3-tiers introduit un serveur intermédiaire en plus des différents composants présents dans l'architecture 2-tiers qui va prendre en charge les différents traitements que doit faire le système. Les données seront, dans ce cas, stockées dans un serveur de données. Nous aurons donc comme architecture générale :

- **Le client** qui va utiliser les services par envoi des requêtes.
- **Le serveur de traitement** qui va prendre en charge tous les traitements réalisés sur les données stockées dans le serveur des données.
- **Le serveur des données** où nous trouvons toutes les données collectées par le système de détection.

La figure 3.12 [URL12] décrit en détail la relation entre les trois couches client/serveur d'application/serveur de données.

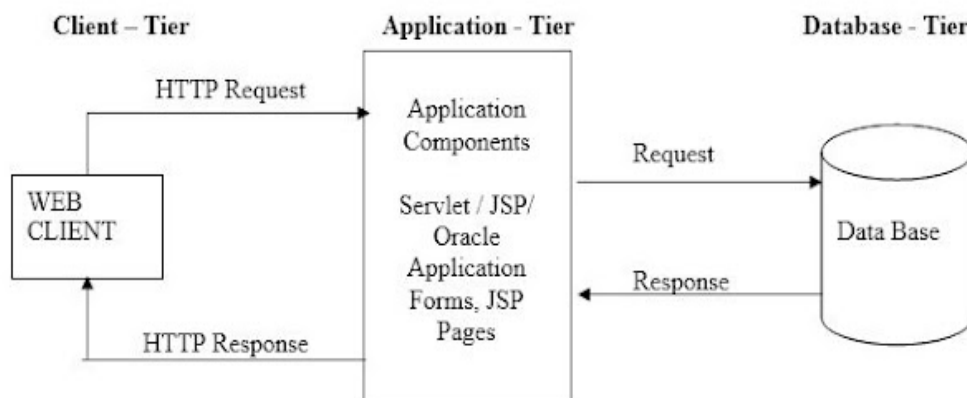


FIGURE 3.12 – Architecture 3-tiers [URL12]

Avantages :

- ✦ Cette architecture présente une facilité au niveau du déploiement.
- ✦ Cette architecture est sécurisée vue que le schéma de la base des données n'est pas exposé directement grâce à l'introduction de la couche de traitement.
- ✦ La manipulation des données est indépendante du support physique de stockage.

Inconvénients :

- L'augmentation du nombre des couches augmente le risque des erreurs.
- Le coût de mise en place du système sera relativement plus élevé.

3.2.2.3 Synthèse et choix :

L'architecture à deux niveaux est donc une architecture client/serveur dans laquelle le serveur est polyvalent, c'est-à-dire qu'il est capable de fournir directement l'ensemble des ressources demandées par le client. Dans l'architecture à trois niveaux, par contre, les applications au niveau serveur sont délocalisées, c'est-à-dire que chaque serveur est spécialisé dans une tâche (serveur Web/serveur de base de données par exemple). Ainsi, l'architecture à trois niveaux permet :

- ✦ une plus grande flexibilité/souplesse
- ✦ une plus grande sécurité (la sécurité peut être dénie pour chaque service)
- ✦ de meilleures performances (les tâches sont partagées)

Il est remarquable que les architectures modernes reposent sur les modèles distribués. Celle-ci favorise une gestion meilleure et une maintenance plus simple. Ainsi nous optons pour l'architecture 3-tiers.

3.2.3 Conception détaillée

Dans la partie précédente, nous définissons bien l'architecture que nous adoptons pour notre projet. Nous devons maintenant formaliser un modèle de données convenable à notre système. Il s'agit de définir le diagramme de classe selon la notation UML comme représenté sur la figure 3.13.

Nous définissons 6 classes décrits comme suit :

- **Classe "Utilisateur"** : Cette classe représente tout utilisateur qui va avoir accès à notre système. Chaque utilisateur va être identifié par un nom d'utilisateur et un mot de passe. Il peut exécuter les opérations suivantes :
 - **S'authentifier()** : Accéder à sa session
 - **Accéder Meteo()** : Suivre les données météorologiques d'une zone donnée
 - **Scan fires()** : Afficher les feux éventuels dans une zone donnée
 - **Afficher Graph()** : Afficher le graphe des données essentielles à la prédiction des feux.
 - **Recevoir Alerte()** : Envoyer une alerte à l'utilisateur suite à la détection des incendies indiquant leur présence.
- **Classe "Agriculteur"** : C'est une classe fille de la classe "Utilisateur" de laquelle elle hérite tous les attributs et méthodes déjà déclarés et définit, en plus, la méthode suivante :
 - **Gérer compte()** : Permettre à l'utilisateur de créer et modifier ces données personnelles
- **Classe "Agent de l'ONPC"** : C'est une classe fille de la classe "Utilisateur" de laquelle elle hérite tous les attributs et méthodes déjà déclarés et définit, en plus, la méthode suivante :
 - **Voir données Region()** : Cette méthode permet d'accéder aux données de tous les agriculteurs dans la région.
- **Classe "Région"** : Cette classe représente les différentes régions susceptibles d'être suivies par notre système.
- **Classe "Champ"** : Cette classe représente les différents champs supervisés par le système. Chaque champ est identifié principalement par sa position. Elle en plus identifiée par l'agriculteur propriétaire de ce champ et sa région.
- **Classe "Incendie"** : C'est l'ensemble des incendies annoncés par le système. Chaque incendie est identifié par sa date d'occurrence et la superficie endommagée et par le champ où l'incendie a pris lieu. Il peut exécuter les opérations suivantes :
 - **Déclencher alerte()** : Cette opération sera appelée chaque fois qu'il y a création d'une nouvelle instance de la classe incendie alertant l'agriculteur à la présence de feux dans ses récoltes.

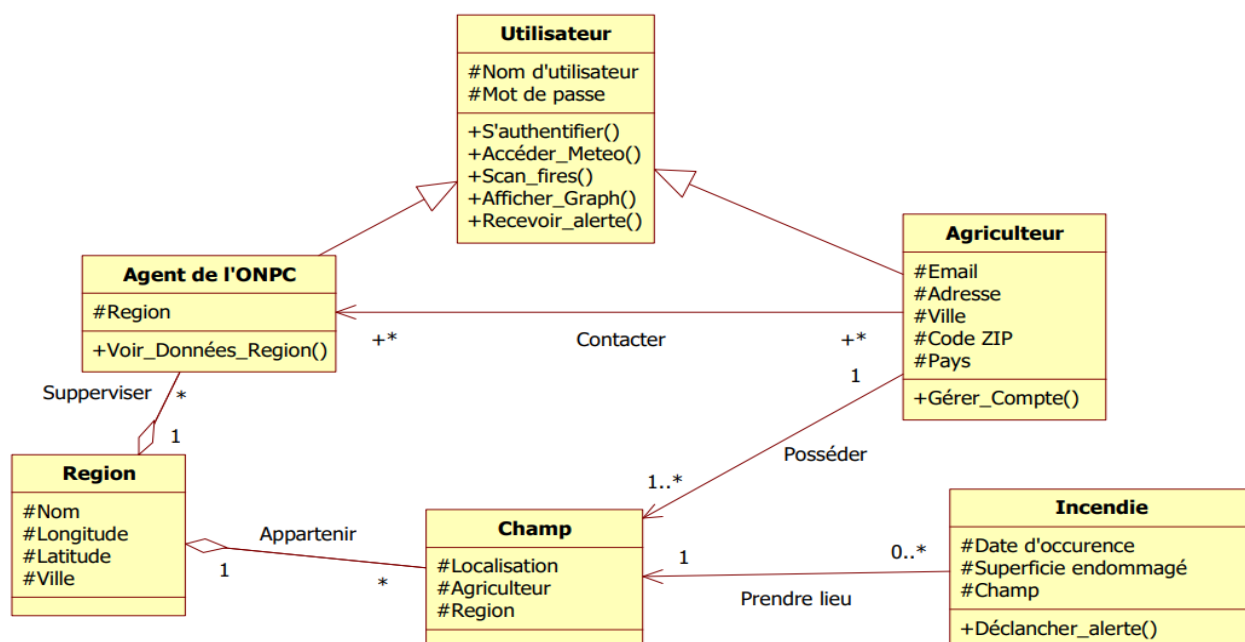


FIGURE 3.13 – Diagramme de classe

3.2.4 Choix technologiques

Afin de pouvoir implémenter les architectures déjà évoquées, nous discutons, dans cette section, les différents choix technologiques qui vont nous permettre de réaliser ce projet.

3.2.4.1 Serveur des données : PostgreSQL V. 8.2

PostgreSQL est un système de gestion de base de données relationnelle orienté objet puissant et open source qui est capable de prendre en charge en toute sécurité les charges de travail de données les plus complexes. Alors que MySQL donne la priorité à l'évolutivité et aux performances, Postgres donne la priorité à la conformité et à l'extensibilité SQL. Contrairement aux autres bases de données transactionnelles, PostgreSQL est implémenté sur un seul serveur et n'est généralement pas conçu pour distribuer ses fonctions de stockage ou de calcul sur plusieurs nœuds.[URL13]

3.2.4.2 Serveur de traitement et composante web : Framework Django

Django est un framework Python de haut niveau. Il permet de développer des sites webs d'une manière rapide et efficace. Créé par des développeurs expérimentés, il prend en charge la plupart des tâches du développement web. Les avantages principaux des applications web créées à travers ce framework et qui vont permettre de se distinguer de tout autre framework sont :

- **la complétude** : Django fournit presque tout ce que les développeurs pourraient vouloir faire. Comme tout ce dont vous avez besoin est une partie de ce "produit", tout fonctionne parfaitement ensemble, offrant une grande cohésion fonctionnelle.

- **la polyvalance** : Django peut être utilisé pour créer presque tous les genres de sites et il peut même fonctionner avec n'importe quelle infrastructure coté client
- **la sécurité** : Django aide les développeurs à éviter les erreurs de sécurité classique
- **la scalabilité** : Django utilise une architecture composite "shared-nothing" ou chaque composant de l'architecture est indépendant de tout autre composant, et peut ainsi être remplacé ou changé si besoin. La séparation nette entre les différentes parties nous permet de se passer à l'échelle lors d'une hausse de trafic en ajoutant du hardware à tous les niveaux.
- **la maintenabilité** : Les principes de conception de Django poussent à la création d'un code représentant la simplicité à maintenir et la réutilisabilité.
- **la portabilité** : Vu qu'il est écrit en python, Django peut fonctionner sous divers plateformes. Cela veut dire que nous ne serons plus contraint par une plateforme en particulier et nous pourrions faire fonctionner nos applications sous autant de versions de Linux, Windows et Mac OS X que nous souhaitons.[URL9]

Grâce à ces différents avantages, nous optons à travailler avec ce framework.

Conclusion

La phase de conception nous permet de faire le choix d'une architecture en trois tiers centrée sur les données. Cette architecture sera implémentée lors de la phase de réalisation. En plus, la conception a aboutit à un modèle logique représenté par le diagramme de classe. Ce diagramme va définir le schéma de notre base de données et ses principales relations.

Chapitre 4

Réalisation

Dans cette partie, nous nous intéressons à la réalisation de notre système. Nous rappelons que l'objectif principal de ce projet est la prédiction des feux de végétation assurée par un modèle d'apprentissage prédictif et une interface de type dashborad qui permet l'interaction avec l'utilisateur et la génération des alertes en cas de feu confirmé. Afin de détailler les étapes de réalisation de cet objectif, nous commençons par la description de l'environnement logiciel, matériel et l'ensemble des technologies utilisées ; ensuite nous nous focalisons sur la réalisation du modèle et des différentes interfaces obtenues et nous terminons par un chronogramme affichant l'avancement de la réalisation du projet.

4.1 Environnement de travail et technologies utilisées

Dans cette section, nous commençons par décrire l'environnement matériel et logiciel.

4.1.1 Environnement matériel

Pour la réalisation de ce projet, nous utilisons principalement les caractéristiques matérielles suivantes :

- **Processeur** : 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz
- **Mémoire vive** : 24.0 GB
- **Carte Graphique** : NVIDIA GeForce RTX 3050 Laptop GPU
- **Stockage** : SSD CL4-3D512-Q11 NVMe SSSTC 512GB

4.1.2 Environnement logiciel

- **Système d'exploitation** : Windows 11 Pro
- **Langage de programmation(Modèle)** : Python
- **IDE** : Visual Studio Code V. 1.88.1
- **Environnement de développement** : Jupyter Notebook
- **Librairies et Framework** :
 - **Pandas** : La gestion des dataframes

- **Numpy** : Les opérations mathématiques et traitement des matrices
- **Matplotlib et Seaborn** : Visualisation des données et tracé des courbes ainsi des diagrammes et histogrammes
- **Django** : Un framework Python qui va permettre de réaliser l'interface Web
- **Optuna** : Un framework open-source qui automatise l'optimisation des hyper-paramètre
- **Imblearn** : Une bibliothèque qui fournit les outils nécessaire pour manipuler les jeux des données non équilibrés
- **Xgboost** : Une "gradient boosting" bibliothèque optimisée distribué créer d'une manière à être efficace, flexible et portable

4.2 Implémentation et Étude des Performances des Modèles de Prédiction

Dans cette partie, nous nous intéressons à l'exploration des performances de chaque modèle implémenté et l'établissement de notre modèle final. Suite à l'étude préliminaire réalisé au premier chapitre, nous choisissons de réaliser l'étude sur principalement les modèles de regression logistique, KNN et SVM et nous analysons leurs performances respectives afin de décider du meilleur modèle de prédiction à retenir pour notre application.

4.2.1 Importation des librairies :

Pour implémenter correctement nos modèles de prédiction et pouvoir évaluer leurs performances, nous importons, tout d'abord, nos modèles de base, les métriques nécessaires pour l'évaluation et les bibliothèques nécessaires pour la visualisation et les représentations graphiques, comme indiqué sur la figure 12.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_validate
from sklearn.model_selection import ShuffleSplit
import time
from sklearn import metrics
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
```

Listing 12 – Importation des librairies

4.2.2 Etude du modèle de Régression logistique

Après avoir entraîné le modèle, nous faisons recours au principe de la validation croisée afin d'obtenir un indice de stabilité et avoir une meilleure généralisation. Les valeurs des métriques d'évaluation obtenues sont comme suit :

```
the accuracy of the model1 is equal to 0.6357879234167892
the precision is equal to 0.6493506493506493
the recall is equal to 0.6006006006006006
Stability Index: 0.015911520313240668
```

Pour améliorer davantage ces résultats obtenus, nous procédons à l'ajustement des hyperparamètres comme discuté dans la section 3.1.3.3 du chapitre 3.

Notons que le plus important hyperparamètre de ce modèle est le facteur de régularisation C qui ne doit pas être assez élevé pour qu'il puisse contrôler la complexité du modèle et réduire le risque de sur-ajustement. Notons que par rapport à l'implémentation de l'Optuna décrite dans 10, nous avons défini la distribution du facteur de régularisation C et notre modèle de régression logistique.

```
LogisticRegression_c= trial.suggest_float('LogisticRegression_c', 1e-30, 2
, log=False)
classifier_obj = LogisticRegression(C=LogisticRegression_c)
```

```
Best trial: 1
Best accuracy: 0.6450662739322534
Best hyperparameters: {'LogisticRegression_c': 1.9014286128198323}
```

Listing 13 – Ajustement des hyperparamètres du modèle de régression logistique

```
the accuracy of the model1 tuned is equal to 0.6450662739322534
the precision is equal to 0.6493506493506493
the recall is equal to 0.6006006006006006
Stability Index: 0.015378616218495691
```

Les résultats obtenus après ajustement sont légèrement différentes des résultats précédents ce qui nous laisse à déduire que, pour la régression logistique, le réglage des hyperparamètres n'a pas un grand effet sur ses performances car

```

from sklearn.calibration import calibration_curve
from sklearn.calibration import CalibratedClassifierCV
model1calibrated=CalibratedClassifierCV(model1tuned,cv=10,method='sigmoid')
train_the_model(model1calibrated)
get_the_score_of_the_model(model1calibrated)
prob = model1tuned.predict_proba(datatest)[: , 1]
prob1=model1calibrated.predict_proba(datatest)[: ,1]
fraction_of_positives, mean_predicted_value = calibration_curve(targettest, prob, n_bins=10)
fraction_of_positives1, mean_predicted_value1= calibration_curve(targettest, prob1, n_bins=10)
plt.figure(figsize=(10, 8))
plt.plot(mean_predicted_value, fraction_of_positives, marker='o', label='model', color='b')
plt.plot(mean_predicted_value1, fraction_of_positives1, color='r', marker='o', label='calibrated')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Perfectly calibrated')
plt.xlabel('Mean predicted probability')
plt.ylabel('Fraction of positives')
plt.title('Calibration Curve')
plt.legend()
plt.show()

```

```

the accuracy of the model1calibrated is equal to 0.6450662739322534
the precision is equal to 0.6493506493506493
the recall is equal to 0.6006006006006006
Stability Index: 0.016633426561592304

```

Listing 14 – Étalonnage du modèle 1

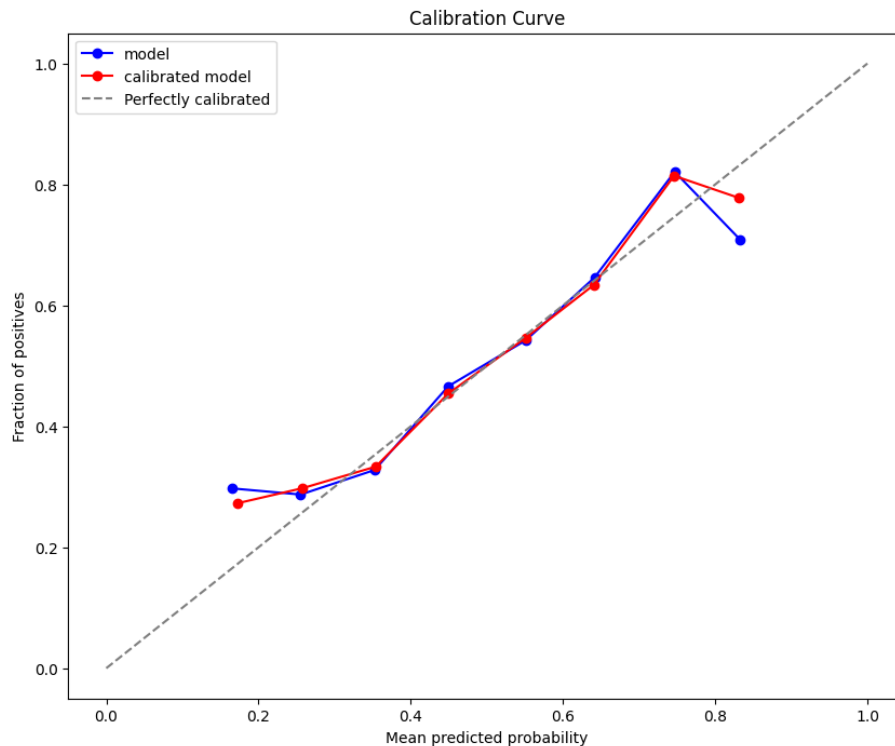


FIGURE 4.1 – Courbe d'étalonnage du 1er modèle

4.2.3 Étude du Modèle de Classification SVM

L'évaluation du modèle de classification SVM nous permet de recenser les résultats suivants :

```
the accuracy of the model is equal to 0.6701030927835051
the precision is equal to 0.7088122605363985
the recall is equal to 0.5555555555555556
Stability Index: 0.030300745993914132
```

Listing 15 – Évaluation du modèle SVC

Pour mieux affiner les résultats d'évaluation, déjà satisfaisants, de ce modèle, nous procédons à l'ajustement des hyperparamètres comme discuté dans la section 3.1.3.3 du chapitre 3. Les hyperparamètres cibles de ce modèle sont :

- **Coefficient de régularisation** : Une valeur élevée signifie une marge d'erreur étroite et d'où une meilleure adaptation aux données d'entraînement mais peut entraîner un sur-ajustement.
- **Gamma** : Un paramètre du noyau RBF, une valeur très élevée peut entraîner un sur-ajustement. Pour notre modèle, nous ajustons la valeur du gamma sur 10.

- **Kernel** :Le noyau détermine la transformation et la séparation des données dans un espace de dimension supérieure. Il y a 3 types de noyaux **linear**, **polynomial**, **RBF** que nous choisisons pour notre modèle.

Notons que de meme pour la regression logistique, nous avons modifié le code d'implémentation 10 en définissant notre modèle SVC et les distributions des hyperparamètres.

```
C = trial.suggest_float('c', 2, 9 ,log=True)
kernel=trial.suggest_categorical('kernel',["linear","poly","rbf","sigmoid"])
classifier_obj = SVC(C=C,kernel='rbf',gamma=10)
```

```
Best trial: 14
Best accuracy: 0.8203240058910162
Best hyperparameters: {'c': 7.445502617990037, 'kernel': 'rbf'}
```

Listing 16 – Ajustement des hyperparamètres du modèle SVC

```
the accuracy of the model is equal to 0.8203240058910162
the precision is equal to 0.8528428093645485
the recall is equal to 0.7657657657657657
Stability Index: 0.02019549620160435
```

Après avoir ajusté les hyperparamètres choisis, nous obtenons les résultats suivants qui sont largement meilleurs qu'en utilisant les valeurs natives.

```
from sklearn.calibration import CalibratedClassifierCV
model2calibrated = CalibratedClassifierCV(model2tuned, method='isotonic', cv=3)
train_the_model(model2calibrated)
get_the_score_of_the_model(model2calibrated)
```

```
the accuracy of the model2 calibrated is equal to 0.8571428571428571
the precision is equal to 0.8314606741573034
the recall is equal to 0.8888888888888888
Stability Index : 0.024834793610470897
```

Listing 17 – Etalonnage du modèle2

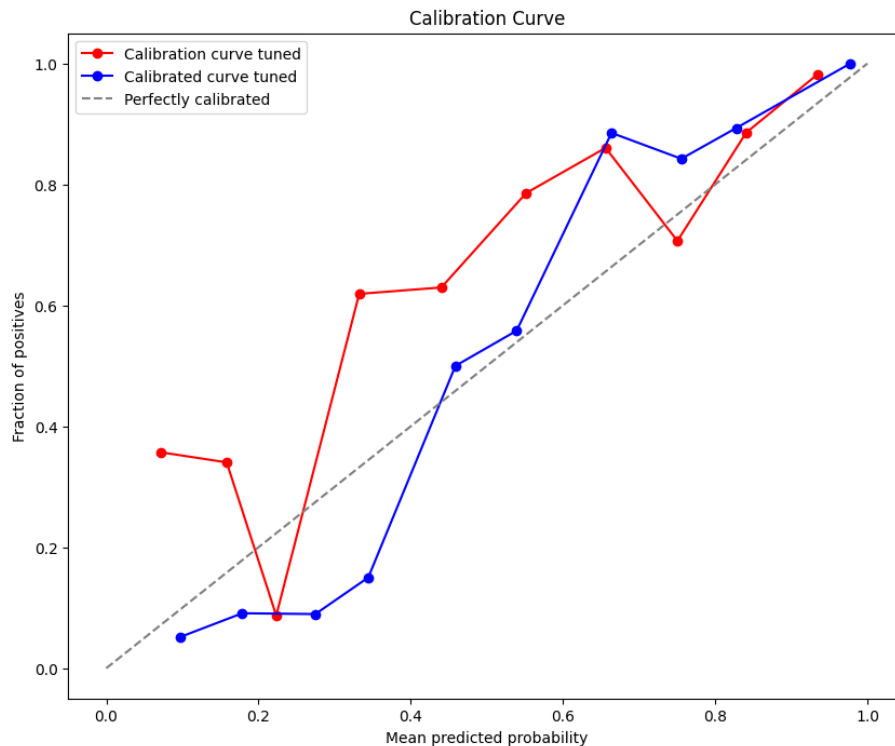


FIGURE 4.2 – Courbe d'étalonnage du 2ème modèle

4.2.4 Etude du modèle KNN

L'entrainement du modèle KNN par validation croisée, nous donne les résultats suivants :

```
the accuracy of the model is equal to 0.7746686303387335\\
the precision is equal to 0.8488372093023255\\
the recall is equal to 0.6576576576576577\\
Stability Index: 0.02034197193598241\\
```

La prochaine étape consiste à trouver la combinaison des hyperparamètres optimales aboutissant aux meilleures performances du modèle SVM. Les hyperparamètres cibles du modèle KNN sont :

- **Nombre des voisins K :**
- **Algorithm :**
- **Distance (p)**

Dans le code d'implémentation de l'Optuna 10, nous allons définir notre modèle et les distributions de ces hyperparamètres.

```

import optuna
def objective(trial):
    k = trial.suggest_int('K', 2, 10 ,log=False)
    algorithm=trial.suggest_categorical('algorithm',['auto', 'ball_tree',
    'kd_tree', 'brute'])
    p=trial.suggest_int('p',1,2,log=False)
    classifier_obj = KNeighborsClassifier(n_neighbors=k,p=p,algoithm=algorithm)

```

```

Best trial: 1
Best accuracy: 0.7967599410898379
Best hyperparameters: {'K': 2, 'algorithm': 'auto', 'p': 2}

```

```

the accuracy of the model is equal to 0.7967599410898379
the precision is equal to 0.975609756097561
the recall is equal to 0.6006006006006006
Stability Index: 0.016682658668874775

```

Listing 18 – Ajustement des hyperparamètres du modèle KNN

Le calibrage du modèle sur notre dataset, nous permet d’améliorer nos prédictions comme suit :

```

from sklearn.calibration import CalibratedClassifierCV
model3calibrated = CalibratedClassifierCV(model3tuned, method='sigmoid', cv=9)
train_the_model(model3calibrated)
get_the_score_of_the_model(model3calibrated)

```

```

the accuracy of the model is equal to 0.8424153166421208
the precision is equal to 0.834319526627219
the recall is equal to 0.8468468468468469
Stability Index: 0.025100051032871374

```

Listing 19 – Étalonnage du modèle KNN

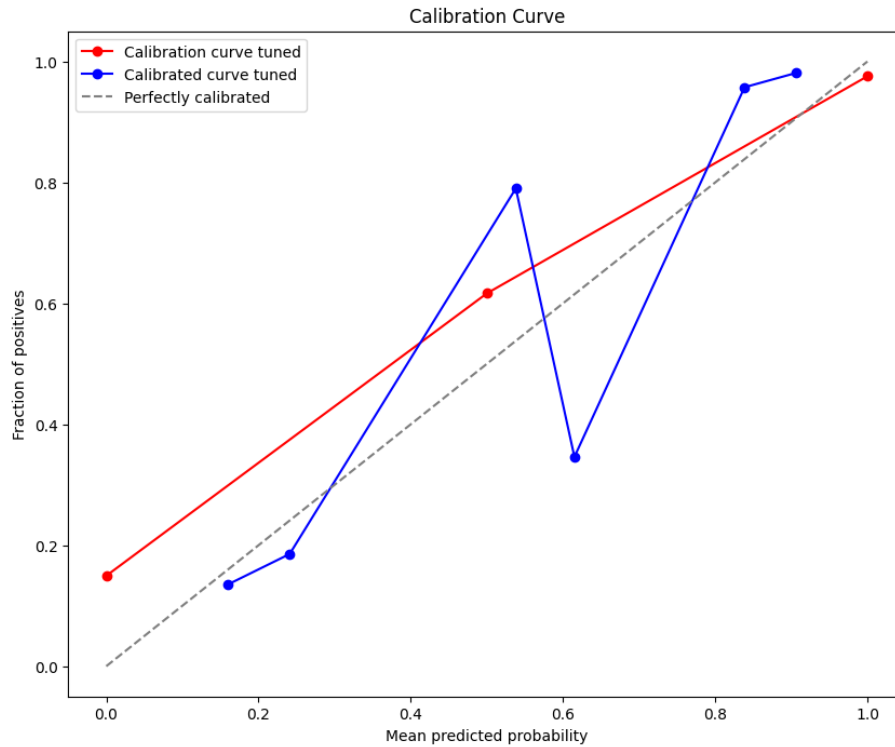


FIGURE 4.3 – Courbe d'étalonnage du 3ème modèle

4.2.5 Synthèse et choix :

Afin d'obtenir un modèle moins biaisé mais surtout avec une variance plus réduite, nous faisons recours à la méthode d'ensemble **stacking**, notre modèle est basé sur trois algorithmes de base répartis sur deux niveaux :

- **Les algorithmes de 1er niveau** : Le 1er niveau est constitué de deux algorithmes : **SVM** et **KNN**. L'entraînement de chaque modèle sur notre *dataset* aboutit à des prédictions représentant les données d'entrée pour le modèle du 2ème niveau.
- **L'algorithme du 2ème niveau** : L'algorithme du 2ème niveau est la **régression logistique** qui, à partir des prédictions fournies précédemment, crée une prédiction finale. Ceci aboutit à une meilleure généralisation.

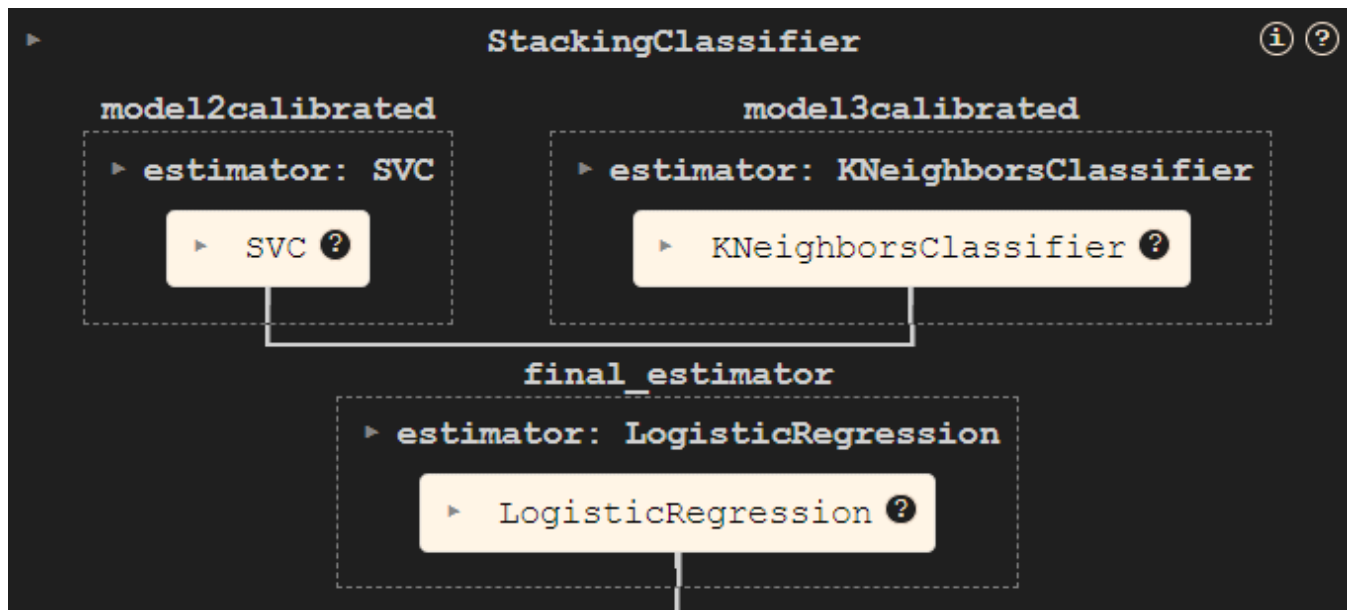


FIGURE 4.4 – Structure du modèle final adopté

Les résultats préliminaires obtenus de ce modèle final que nous définissons sont comme suit :

```

the accuracy of the model is equal to 0.8497790868924889
the precision is equal to 0.8553846153846154
the recall is equal to 0.8348348348348348
Stability Index: 0.025347590992026083

```

```

the accuracy of the finalmodelcalibrated is equal to 0.8409425625920471
the precision is equal to 0.8419452887537994
the recall is equal to 0.8318318318318318
Stability Index: 0.025067896052860812

```

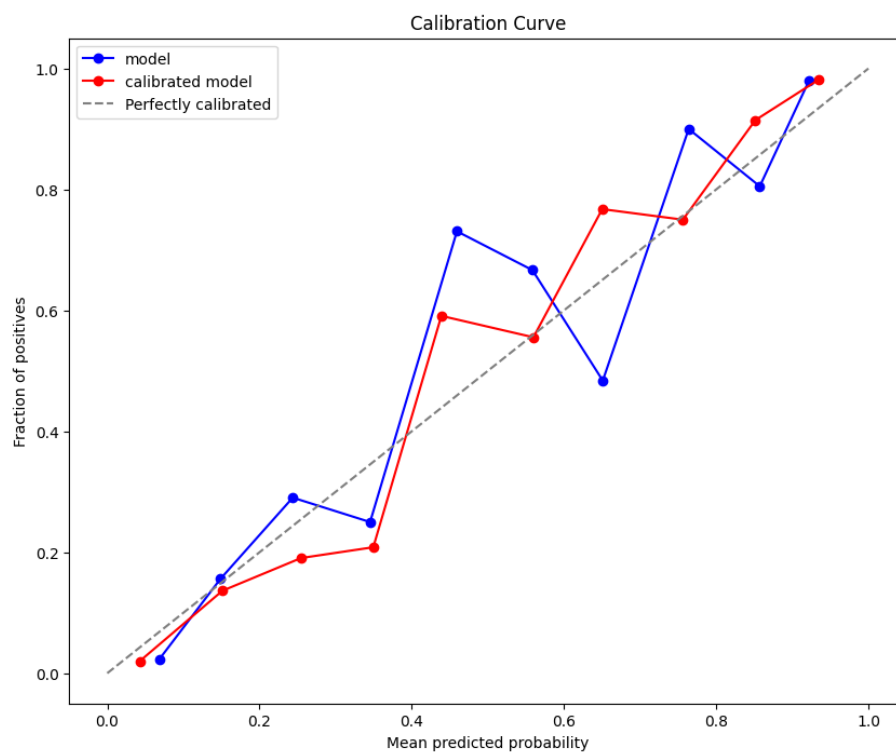


FIGURE 4.5 – Courbe d'étalonnage

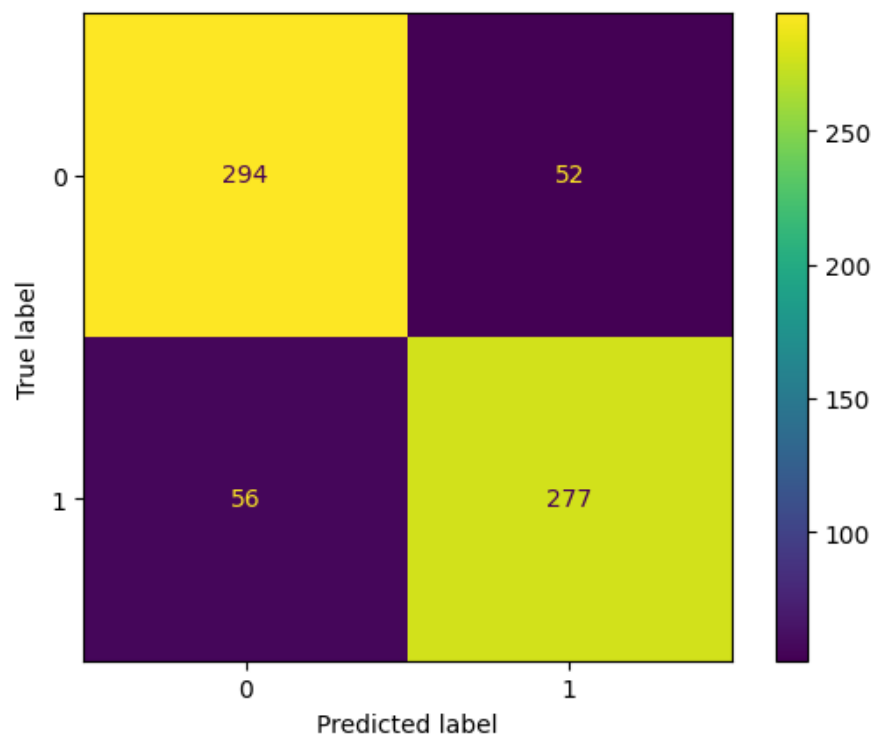


FIGURE 4.6 – Matrice de confusion

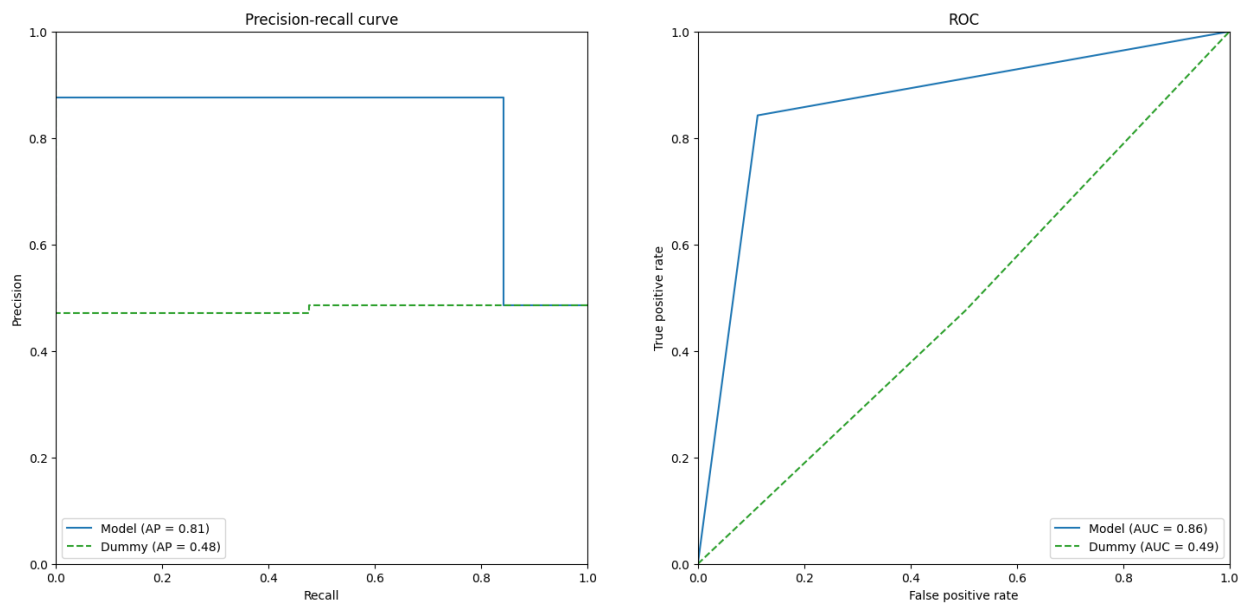


FIGURE 4.7 – Interprétation graphique des résultats de performances

4.3 Implémentation du tableau de bord

Cette section décrit les différentes interfaces développées pour offrir à l'utilisateur l'interaction avec le système de prédiction des feux de végétation et ce, selon l'analyse et la conception élaborées dans les chapitres précédents.

4.3.1 Interfaces communes à tous les utilisateurs

4.3.1.1 Interface d'authentification

La page d'authentification 4.8 est la première interface de notre système. Elle permet à l'utilisateur de s'authentifier en saisissant son login et mot de passe afin d'accéder aux autres fonctionnalités du système. En cas d'erreur d'authentification, un message sera affiché comme dans la figure 4.9 et l'utilisateur sera invité à corriger l'erreur.

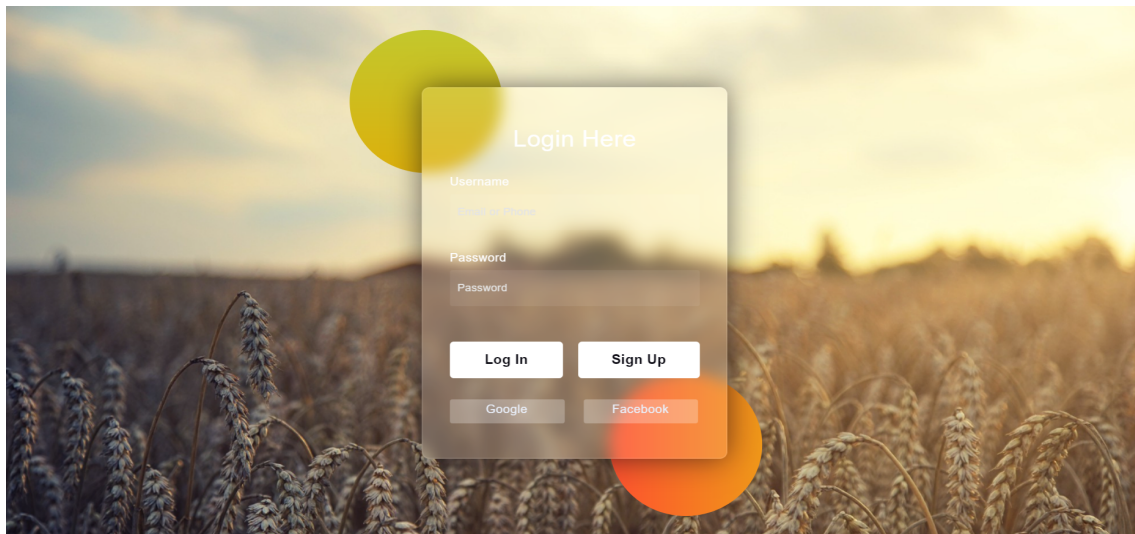


FIGURE 4.8 – Interface d’authentification

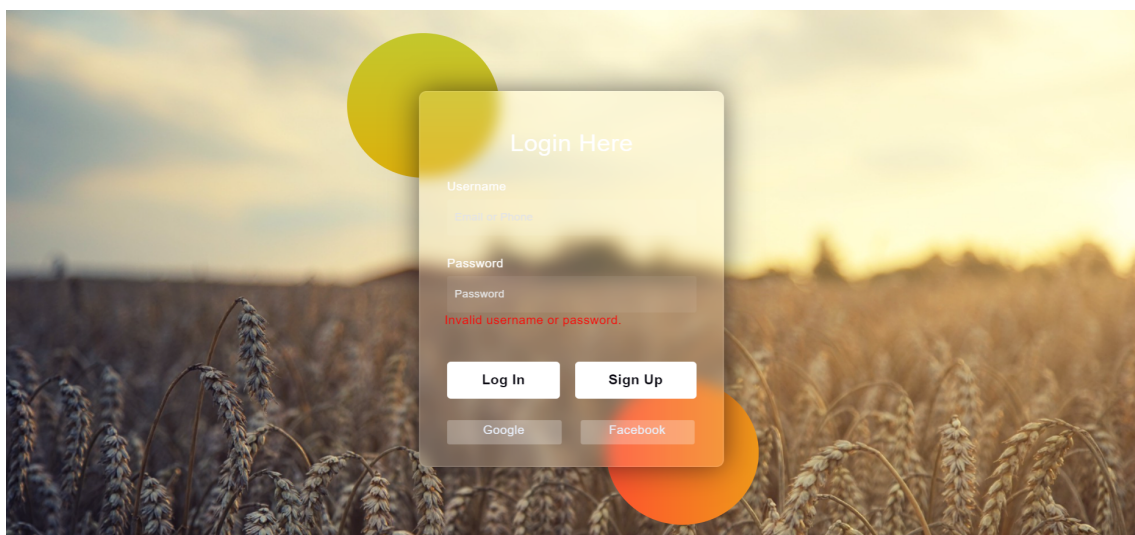


FIGURE 4.9 – Erreur d’authentification

4.3.2 Interfaces spécifiques à l’agriculteur

- **La page d’accueil de l’agriculteur** : Suite à une bonne authentification, l’agriculteur va avoir accès à son espace d’accueil présenté par la figure 4.10. A travers cete interface, il va avoir accès aux différentes fonctionnalités offertes par ce système, à savoir :
 - Accéder aux données météorologiques de la position actuelle. Un agriculteur peut se renseigner sur d’autre zones
 - Afficher un Graphe contenant un historique complet sur les valeurs de trois grandeurs principales et essentielles à la prédiction des feux (NDVI, LST et BURNED AREA)

- Accéder à une carte donnant sa position actuelle
- Afficher un bouton "Scan for fires". Si un agriculteur va appuyer sur ce bouton, l'interface va afficher les zones ou les feux vont être présent en fonction des valeurs des grandeurs caractéristiques de son champ.

Dans le cas de détection d'un incendie, l'agriculteur sera notifié sur ce dashboard avec l'affichage d'une icône et d'un texte correspondant dans un cadre rouge comme l'indique la figure ??

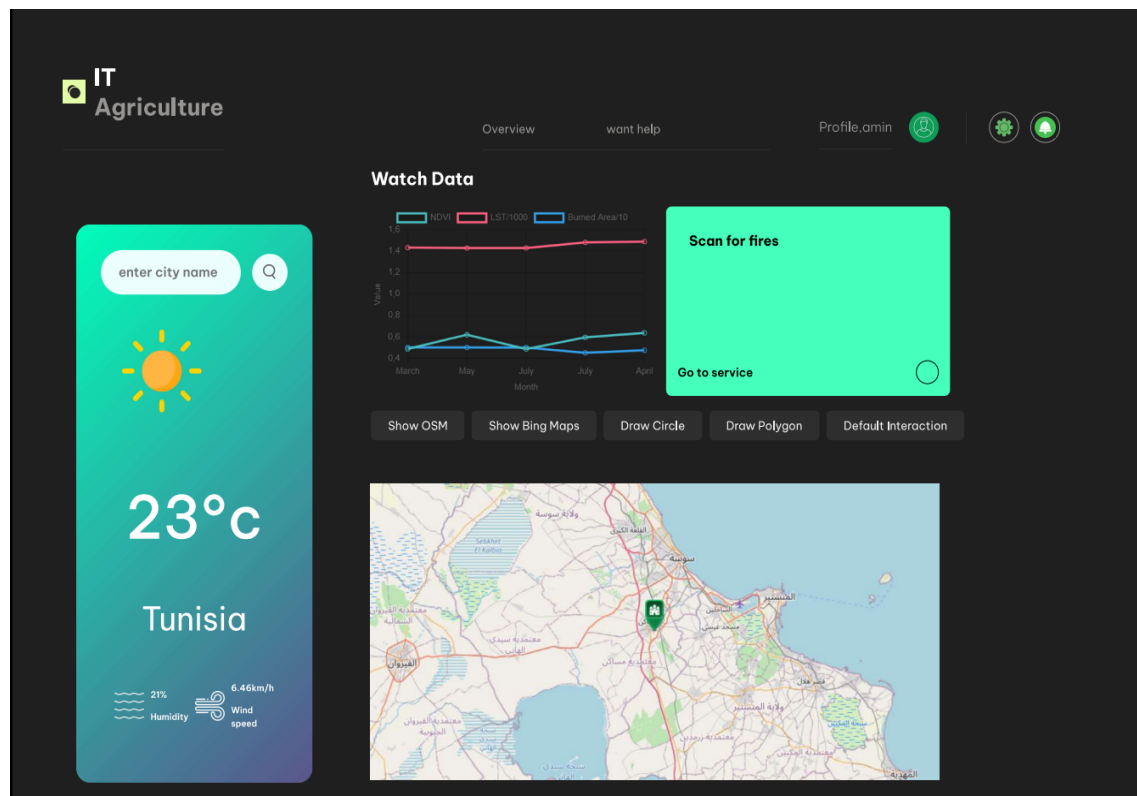


FIGURE 4.10 – Interface d'accueil de l'agriculteur

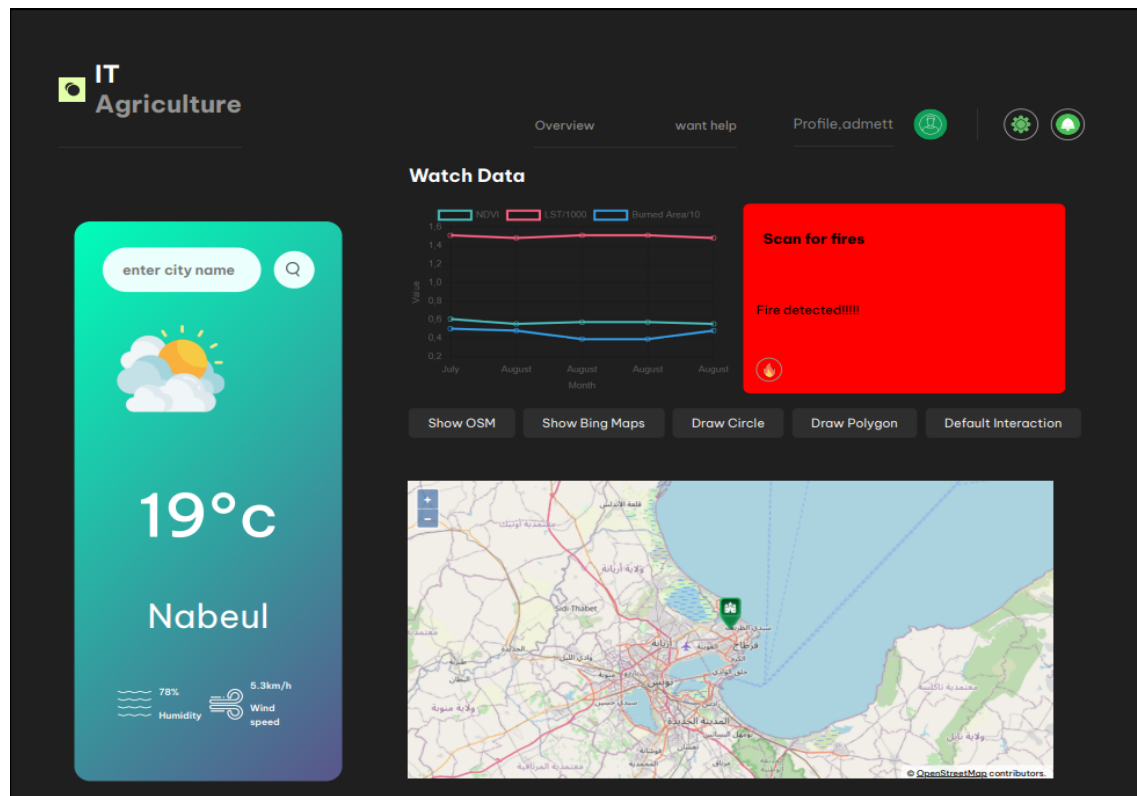


FIGURE 4.11 – Interface d'accueil de l'agriculteur dans le cas détection d'un incendie

- **Interface de gestion de compte** : Cette page sera accessible seulement aux agriculteurs et va leur permettre de créer un nouveau compte et de modifier leurs données personnelles. La figure 4.12 illustre le formulaire d'ajout/modification du compte.

sign in page

your username:	Street Address:
username	Address
your password:	city:
password	city
your email:	Country
email	
your profession:	Zip Code:
Agriculture	zip Code
Submit	

FIGURE 4.12 – Interface de gestion de compte

4.3.3 Interfaces spécifiques à l'agent de l'ONPC

- **Interface d'accueil de l'agent de l'ONPC** : Suite à une bonne authentification, l'agent de l'ONPC va avoir accès à son dashboard présenté par la figure 4.13. A travers cette interface, il va avoir accès aux différentes fonctionnalités offertes par ce système, à savoir :
 - Accéder aux données météorologiques de sa région. Un agent de l'ONPC peut se renseigner sur d'autres zones
 - Afficher un Graphe contenant un historique complet sur les valeurs de trois grandeurs principales et essentielles à la prédiction des feux relative à sa région (NDVI, LST et BURNED AREA)
 - Accéder à une carte donnant la position actuelle des agriculteurs localisés dans sa région
 - Afficher un bouton "Scan for fires". Si un agent de l'ONPC va appuyer sur ce bouton, l'interface va afficher les zones où les feux vont être présents en fonction des valeurs des grandeurs caractéristiques de la région et des agriculteurs qui vont être affectés par ce feu.

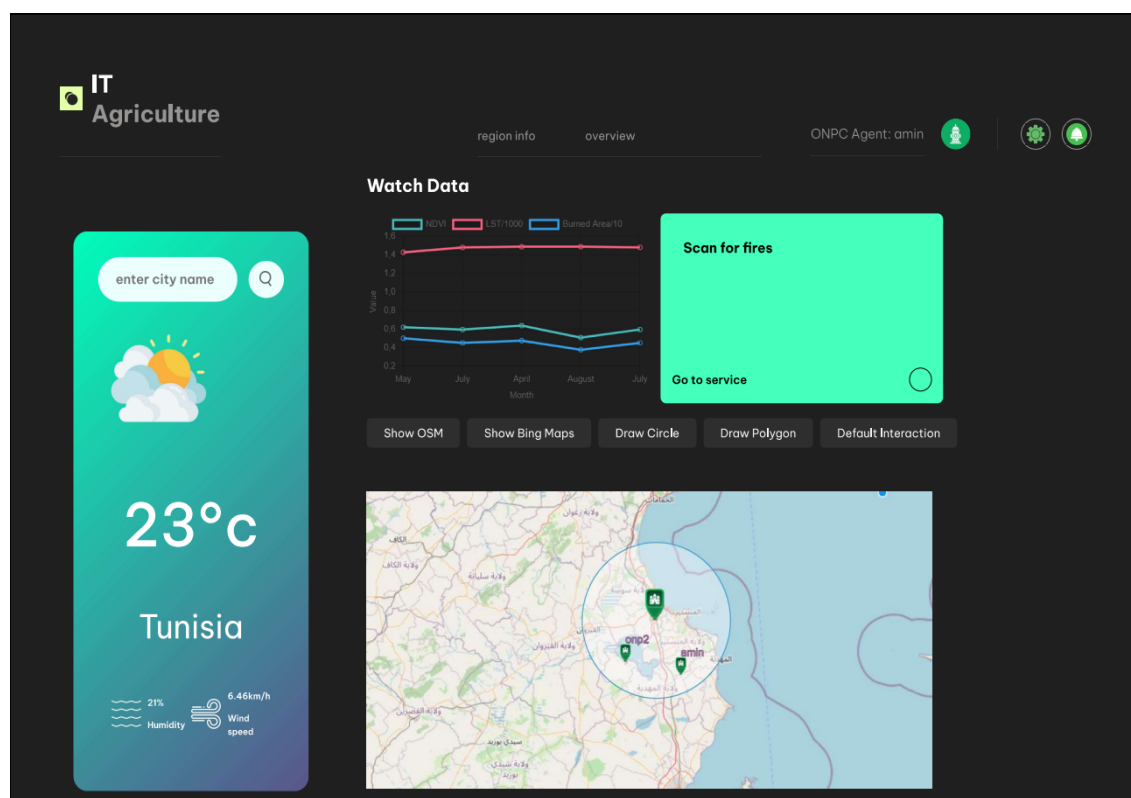


FIGURE 4.13 – Interface d’accueil de l’agent de l’ONPC

- **Interface d’investigation et d’historique** : Cette page est exclusive aux agents de l’ONPC. Elle permet d’afficher les données relatives aux agriculteurs localisés dans une région d’intérêt. La figure 4.14 affiche l’ensemble des informations qu’un agent peut y avoir accès.

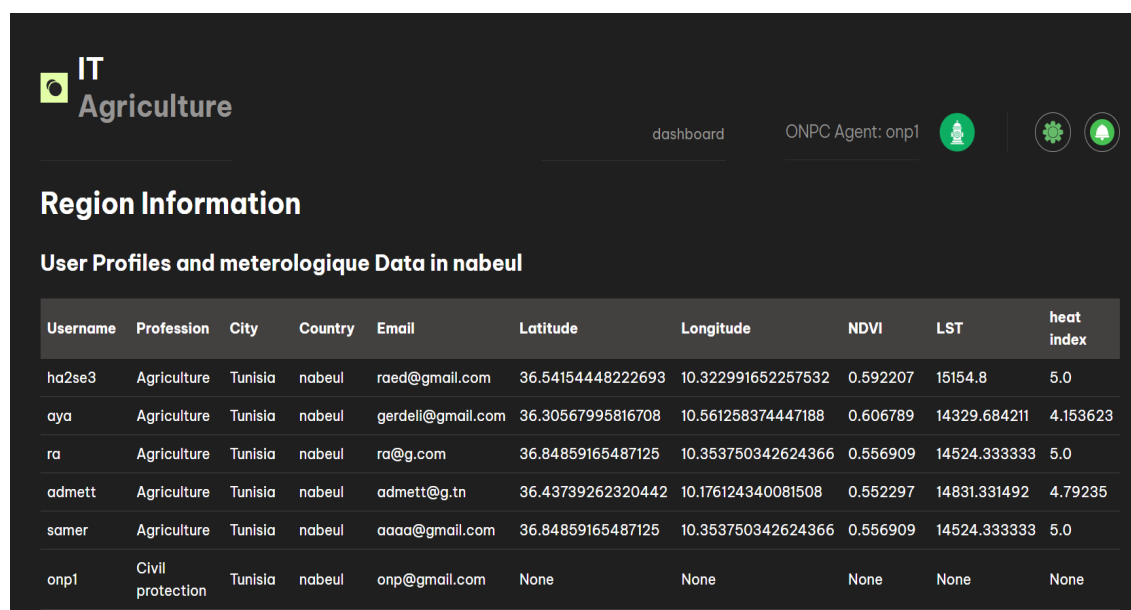


FIGURE 4.14 – Interface d’investigation et d’historique

4.4 Chronogramme

La réalisation de ce projet a débuté le 01 Février 2024 et s’est achevé vers le 05 Mai 2024. Nous commençons tout d’abord par une étude approfondie de l’existant avant de commencer le traitement du sujet. Il s’agit principalement d’une exploration des enjeux liés au contexte de notre travail, d’une recherche minutieuse des jeux des données adéquats et d’une étude succincte des modèles de prédiction pouvant être utilisés. Ensuite, nous menons une analyse détaillée de nos besoins fonctionnels et non fonctionnels pour déboucher sur une conception à la fois logicielle et données. La réalisation de ce projet a été menée en dernier lieu et la rédaction du rapport s’est faite en parallèle avec toutes ces étapes. Afin de pouvoir mieux comprendre la répartition temporelle des différentes tâches, nous établissons le chronogramme de la figure 4.15 suivante.

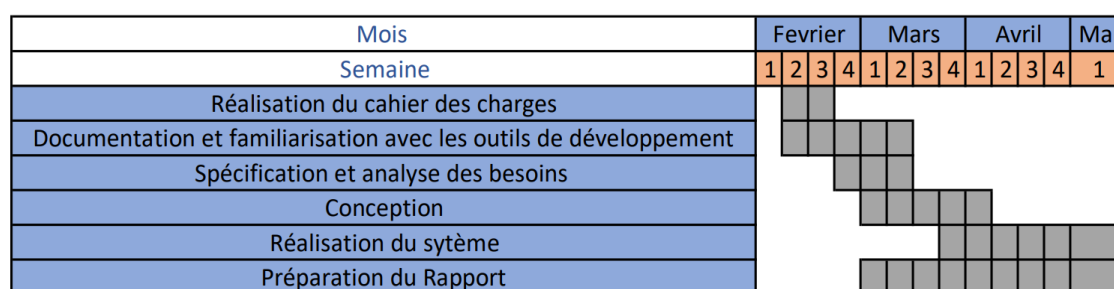


FIGURE 4.15 – Chronogramme de travail

Conclusion

Dans ce chapitre, nous listons l'ensemble des choix technologiques pour la réalisation de ce projet. Ensuite, nous détaillons l'implémentation et l'évaluation des performances des différents modèles de prédiction étudiés. Puis, nous ajoutons l'ensemble des captures d'écran résumant le fonctionnement de notre système. Enfin, nous terminons avec un chronogramme résumant notre avancement dans le temps.

Conclusion Générale et Perspectives

Le travail présenté dans ce présent rapport, a été élaboré dans le cadre du Projet de Conception et de Développement (PCD) de notre cycle d'ingénieur.

Dans le cadre de ce projet, nous avons eu l'occasion de développer un système de prédiction des feux de végétation en utilisant les techniques de Machine Learning.

Pour atteindre cet objectif, nous avons suivi une méthodologie de travail incrémentale commençant par l'établissement d'un cahier de charges détaillé, suivi par l'élaboration de l'ensemble des besoins fonctionnels et non-fonctionnels de notre système. Ensuite, nous avons réalisé une conception détaillée tant de l'aspect données que de l'aspect logiciel du système ce qui nous a facilité l'étape de la réalisation et de l'implémentation du système.

Durant la réalisation de ce projet, nous avons pu enrichir nos compétences en relation avec les différents choix technologiques faits tout au long de ce projet. Ainsi, nous avons pu maîtriser le fonctionnement du framework Django et les différentes options qu'il peut offrir pour la conception et la réalisation des sites web. En plus, nous avons eu l'opportunité de comprendre encore le fonctionnement des différents modèles de Machine Learning et tout ce qui se rapporte au traitement des données.

Lors du développement de ce système, nous nous sommes aussi confrontés à plusieurs difficultés qui ont affecté l'avancement de la réalisation de ce système. Le premier défi qui s'est posé est le manque de données correspondant à nos besoins spécifiques. En effet, la plupart des jeu de données disponibles sont soit des images de forêts et donc ne sont pas fidèles aux caractéristiques des champs de végétation, soit de mauvaise qualité (trop d'erreurs, de valeurs aberrantes et nécessitant ainsi un nettoyage approfondi avant utilisation), soit de taille réduite (nombre réduit d'échantillons). Ce problème a été soulevé suite à une recherche approfondie et minutieuse des différents répertoires et bases des données.

Le deuxième défi auquel nous avons été confrontés est la familiarisation avec les outils de développement particulièrement l'intégration des API de géolocalisation dans l'outil Django. En effet, la plupart des API de géolocalisation nécessitent une clé d'API pour authentifier les requêtes. De plus, les API de géolocalisation peuvent avoir des limites de requêtes gratuites ou des tarifs basés sur l'utilisation, ce qui a limité le nombre de requêtes que nous pouvions effectuer et nous a amené à l'utilisation de bibliothèques spécifiques telle que Google Maps API pour afficher des cartes et des marqueurs géographiques dans notre application. D'autre part, l'interaction avec les API de géolocalisation implique généralement l'envoi de requêtes HTTP

et la gestion des réponses JSON ou XML ce qui nous a poussé à un travail de restructuration pour pouvoir stocker ces données dans la base de données.

Le système proposé reste toujours ouvert à l'amélioration sur différents plans et selon les besoins des différents utilisateurs. Par exemple, la gestion des alertes peut être faite par envoi d'email ou sur Tel pour mieux garantir une intervention rapide des personnes concernées. De plus, le modèle d'apprentissage adopté peut être mieux affiné de sorte à réduire le temps d'évaluation et d'améliorer sa précision. D'autre part, le système pourrait offrir une carte géographique plus précise en indiquant le taux de risque de feu pour chaque zone du champ de végétation.

Bibliographie

- [1] Cruz Miguel G., Hurley Richard J., Bessell Rachel, Sullivan Andrew L. “Fire behaviour in wheat crops – effect of fuel structure on rate of fire spread. International Journal of Wildland Fire”, vol. 29, pp. 258-271, 2020.
- [2] Younes Oulad Sayad, Hajar Mousannif, Hassan Al Moatassime, “Predictive modeling of wildfires : A new dataset and machine learning approach”, Fire Safety Journal, Vol. 104, pp. 130-146, 2019.
- [3] Miao, Xinyu and Jian Li and Yunjie Mu and Cheng He and Yunfei Ma and Jie Chen and Wentao Wei and Demin Gao, "Time Series Forest Fire Prediction Based on Improved Transformer", Forests, vol. 14, No. 8 :1596, 2023.
- [4] T. Curt, C. Hély, R. Barbeo, J.-L. Dupuy, F. Mouillot, J. Ruffault, "Feux de végétation : comprendre leur diversité et leur évolution", Editions CIRAD, INRAE, ISSN : 2267-3032
- [5] Varela, Vassiliki, Diamando Vlachogiannis, Athanasios Sfetsos, Stelios Karozis, Nadia Politi, and Frédérique Giroud, "Projection of Forest Fire Danger due to Climate Change in the French Mediterranean Region" Sustainability 11, no. 16, 2019.
- [6] Fatih Sivrikaya, Ömer Küçük, "Modeling forest fire risk based on GIS-based analytical hierarchy process and statistical analysis in Mediterranean region", Ecological Informatics, Volume 68,2022.

Netographie

[URL1] <https://www.reuters.com/business/environment>, Date de dernier accès : 04/03/2024

[URL2] <https://www.ibm.com/blog/supervised-vs-unsupervised-learning/>, date de dernier accès : 10/4/2024

[URL3] <https://towardsdatascience.com/overview-of-machine-learning-algorithms-regre> date de dernier accès : 8/4/2024

[URL4] <https://optuna.org/>, date de dernier accès : 22/4/2024

[URL5] <https://firms.modaps.eosdis.nasa.gov/>, date de dernier accès : 22/4/2024

[URL6] <https://modis.gsfc.nasa.gov/>, date de dernier accès : 22/4/2024

[URL7] <https://www.clicours.com/svm-principe-de-fonctionnement-general/>, date de dernier accès : 29/4/2024

[URL8] <https://medium.com/@TheDataScience-ProF/understanding-hinge-loss-in-machine> date de dernier accès : 29/4/2024

[URL9] <https://developer.mozilla.org/fr/docs/Learn/Server-side/Django/>, date de dernier accès : 1/5/2024

[URL10] <https://hevodata.com/learn/big-data-pipeline/>, date de dernier accès : 3/5/2024

[URL11] https://csis.pace.edu/~marchese/SE616_New/L6/L6.htm date de dernier accès : 6/5/2024

[URL12] <https://www.oracleapps2fusion.com/2016/03/application-architecture-single-t> html date de dernier accès : 6/5/2024

[URL13] <https://www.oracle.com/fr/database/definition-postgresql/> date de dernier accès : 6/5/2024

[URL14] [https://github.com/ouladsayadyounes/WildFires/blob/master/README.](https://github.com/ouladsayadyounes/WildFires/blob/master/README.md) md, Date de dernier accès : 03/05/2024

[URL 15] <https://www.geeksforgeeks.org/ml-classification-vs-regression/>, date de dernier accès : 8/4/2024

Résumé Dans ce projet, nous proposons un système de prédiction des feux dans les champs de végétation. Ce système permet le suivi en temps réel de données météorologiques et d'informations pertinentes liées à la végétation et, en se basant sur les techniques d'apprentissage automatique, déduit une cartographie de risque capable d'identifier les positions éventuelles des feux déclenchés ainsi qu'une estimation des dommages engendrés. Il fournit également des alertes en cas de risque confirmé de feu ce qui garantit une intervention rapide et efficace des parties concernées. Nous proposons, aussi, de développer un tableau de bord permettant à l'utilisateur d'interagir avec le système, se renseigner et suivre en temps réel l'état de ses champs. Dans le cadre de ce projet, nous proposons une solution développée en temps réel en utilisant le framework Django et en se basant sur les technologies des services web.

Mots clés : Prédiction, Détection, feux de végétation, apprentissage automatique, Django, services web.

Abstract In this project, we propose a system for predicting fires in vegetation fields. This system allows real-time monitoring of weather data and relevant information related to vegetation and, based on machine learning techniques, deduces a risk map capable of identifying the possible positions of fires triggered as well as an estimation of the damage caused. It also provides alerts in the event of a confirmed fire risk, which guarantees rapid intervention and efficiency of the parties concerned. We also propose to develop a dashboard allowing the user to interact with the system, obtaining information and monitoring the status of their fields in real time. As part of this project, we propose a solution developed in real time using the Django framework and based on web services technologies.

Keywords : Prediction, Detection, vegetation fire, machine learning, Django, web services.